POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY 2

----------o0o----------

FINAL PROJECT REPORT

Subject : INTRODUCING TO DEEP LEARNING

Project: **VietLaw QA Chat**

Supervisor:	***Nguyen Ngoc Duy***	
Submitted by:	*Pham Nguyen Quoc Huy*	*N22DCCI018*
	Vu Hoang Phat	*N22DCCN058*
	Tran Phi Hung	*N22DCCN132*
Class:	*E22CQCNTT01-N*	
Term:	*2022-2027*	
Program:	*CLC*	

Ho Chi Minh, 15th June 2026

Table Of Contents

Tables and Figures

SUMMARY[1]

This project builds a complete legal question-answering pipeline for Vietnamese statutory law using a cascaded hybrid retrieval architecture. The system integrates a TextCNN topic classifier (Kim, 2014) for domain prediction, TF-IDF + Siamese BiLSTM (Neculoiu et al., 2016) hybrid retrieval for relevance ranking, and Groq LLM (llama-3.3-70b) for grounded answer synthesis with automatic abstention when retrieved context is insufficient. The entire pipeline is deployed as a single-file web application with zero hallucination — answers are either verified against source text via n-gram overlap checking or the system explicitly refuses to answer. The project comprises 2,065 lines of Python source code, 1,674 lines of experiments and evaluation code, 18 Colab notebooks, 4 data variants (k1–k4) exploring class-merging strategies, and 3 trained models producing production artifacts for both topic classification and dense retrieval. Key results: TextCNN topic classification achieves macro F1 = 0.7602 on the merged 5-class configuration (k4, dropout probability = 0.5), representing a +15.2-point gain over the 8-class baseline. Siamese BiLSTM dense retrieval achieves MRR = 0.377 at maximum sequence length = 512, the sweet spot between insufficient context (256) and LSTM memory-window degradation (1024). Hybrid retrieval combines TF-IDF exact keyword matching (first-pass, top-200 candidates) with Siamese cosine re-ranking, mitigating the Siamese MRR = 0.377 limitation by ensuring high recall for distinctive legal terminology.

[1] Github: hyperformancelabs/vnlegal-rag

TEAM WORK ASSIGNMENT TABLE

NO.	STUDENT ID	LAST NAME	FIRST NAME	TASKS
1	N22DCCI018	Pham Nguyen Quoc	Huy	TF-IDF baseline, TextCNN pipeline (Kim 2014), dropout sweeps, web demo.
2	N22DCCN058	Vu Hoang	Phat	Infrastructure, tokenizer, data prep, Siamese BiLSTM, test suite, report, git.
3	N22DCCN132	Tran Phi	Hung	Contrastive loss, hard negative mining, topic-filter routing, LLM synthesis & grounding.

Chapter 1: Introduction

1.1 Background

Vietnam operates a civil-law legal system in which codified statutes enacted by the National Assembly constitute the primary source of law. These statutes span multiple domains — finance, banking, civil procedure, criminal justice, transportation, labor, environmental regulation — and are routinely consulted by legal professionals, corporate compliance officers, and citizens alike. The sheer volume and frequent amendment of Vietnamese legislation create a pressing need for automated legal question-answering systems that can retrieve and synthesise relevant statutory provisions from a large corpus. Traditional keyword-based search (e.g., Google, Thư viện pháp luật) returns documents matching query terms but does not provide direct answers grounded in specific articles, nor does it reason across multiple documents to produce a coherent response.

The VNLegal dataset (HuggingFace was curated to support research on Vietnamese legal QA. It covers 8 macro-domains of statutory law: (1) Finance & Banking, (2) State Organization & Admin, (3) Justice & Dispute Resolution, (4) Transportation, (5) Labor & Insurance, (6) Civil & Investment, (7) Industry, Resources & Environment, and (8) Security & Defense. Each document corresponds to a full legal statute — for example *Luật Các tổ chức tín dụng 2024* — split at the individual article (*điều*) level into passage-sized units. Accompanying question-answer pairs provide natural-language queries grounded in specific passages, making the dataset suitable for both retrieval and generative tasks. The dataset exhibits a long-tail class distribution: Finance & Banking dominates with 33.7% of passages, while Security & Defense accounts for only 3.0%, motivating the class-merging experiments described in Sections 3.8 and 6.1.

1.2 Problem Motivation

Despite advances in neural retrieval and large language models, building a reliable legal QA system for Vietnamese faces several specific challenges that distinguish it from general-domain QA or English-language legal QA. First, vocabulary mismatch in legal terminology. Vietnamese legal texts employ a controlled but domain-specific lexicon — terms such as *thẩm quyền* (jurisdiction), *tố tụng* (procedure), *khởi tố* (prosecution) — that generic semantic embeddings often fail to discriminate. Prior experiments in this project

confirm that a Siamese BiLSTM dense retriever achieves only MRR = 0.377, meaning the correct statute appears as the top-1 result only 24.5% of the time. Second, class imbalance and label granularity. The Justice & Dispute Resolution category encompasses civil procedure, criminal procedure, administrative procedure, notary, judicial enforcement, and juvenile justice — five or more distinct sub-domains under a single label. The TextCNN topic classifier achieves only 31% recall on this class regardless of hyperparameter tuning, causing criminal-law queries to be misrouted to other domains. Third, corpus coverage gaps. The VNLegal dataset does not include foundational codes such as *Bộ luật Hình sự* (Criminal Code) or *Bộ luật Lao động 2019* (Labour Code), creating an irreducible blind spot for queries about criminal penalties or labour rights. Fourth, hallucination risk in generative QA. Without strict grounding checks, an LLM may fabricate statutory references or misstate legal provisions — a risk that is unacceptable in a legal context where accuracy is paramount. These challenges collectively motivate our design: a cascade of sparse first-pass retrieval (TF-IDF, robust to vocabulary mismatch), dense re-ranking (Siamese BiLSTM, capturing semantic similarity), conservative topic filtering (the topic classifier at confidence ≥ 0.85 only), and LLM answer synthesis with automatic abstention via n-gram overlap verification.

1.3 Objectives

This project aims to build and evaluate a complete retrieval-augmented generation pipeline for Vietnamese statutory law. The specific objectives are as follows. Objective 1 — Data preparation and analysis: Process the raw VNLegal dataset into a structured corpus with clean train-validation-test splits, characterise the class distribution and token-length statistics, and create multiple class-merging variants (k1–k4) to study the effect of label granularity on classification performance. Objective 2 — Topic classification: Implement a Kim-style TextCNN (Kim, 2014) for macro-domain prediction of user queries, with strict fidelity to the original paper's hyperparameters (Adadelta optimiser, L2 max-norm constraint s = 3 on FC weights, Uniform-distributed OOV initialisation matching pretrained variance, no weight decay) and a systematic dropout sweep (0.2, 0.3, 0.5, 0.7) to identify the optimal regularisation configuration. Objective 3 — Dense retrieval: Implement a Siamese BiLSTM (Neculoiu et al., 2016) for pairwise text similarity, adapted from the original 4-layer character-level architecture to a 1-layer word-level architecture after experimental evidence showed signal collapse with deeper stacks. Design a contrastive loss function that accommodates the 4:1 negative-positive ratio of legal QA pairs: $L^+ = 3.0 \cdot (1 - \cos)^2$, $L^- =$

relu(cos − 0.5)², with fixed margin and no annealing after margin = 0.85 was shown to create a dead-zone gradient. Objective 4 — Hybrid retrieval: Combine sparse (TF-IDF) and dense (Siamese) retrieval in a cascaded pipeline — TF-IDF first-pass for top-200 keyword candidates, optional topic-label filtering when TextCNN confidence ≥ 0.85, then Siamese cosine re-ranking — to mitigate the vocabulary-mismatch weakness of pure dense retrieval. Objective 5 — Grounded answer generation: Integrate a Groq LLM (llama-3.3-70b) for answer synthesis, constrained to use only verbatim context from retrieved passages, with automatic abstention enforced via an n-gram overlap verifier (8-word minimum) that rejects hallucinated answers. Objective 6 — Deployable demo: Package the entire pipeline as a single-file HTTP server (using Python standard library) with an inline HTML/CSS/JS interface that accepts Vietnamese legal queries and returns topic predictions, verified answers, and ranked retrieval results.

1.4 Contributions

This project makes the following specific contributions. (1) Word2vec embeddings for Vietnamese legal text with streaming loader. The pretrained 300-dim Vietnamese syllable-level word2vec (979,460-word vocabulary, 3.4 GB flat text) is integrated via a streaming loader that processes the file line-by-line without loading the full matrix into memory. OOV initialisation follows Kim (2014, §4.3): Uniform[−a, a] where $a = \sqrt{(3 \cdot \sigma^2\_w2v)}$, using the empirical variance of the pretrained vectors rather than a standard-normal draw. (2) Vietnamese syllable normalisation. The VinAI Research (2021) diacritic-normalisation dictionary (45 entries covering *oa*, *oe*, *uy* diphthongs across five tones and three positional cases) is integrated into the tokenisation pipeline to standardise legacy orthographic variants before tokenisation, ensuring compatibility with the word2vec vocabulary. (3) Siamese BiLSTM with tailored contrastive loss. A 1-layer Siamese BiLSTM with $L^+ = 3.0 \cdot (1 - \cos)^2$ and $L^- = \text{relu}(\cos - 0.5)^2$ is designed to handle the 4:1 negative-positive ratio of Vietnamese legal QA pairs. Fixed margin = 0.5 avoids the gradient dead-zone empirically observed with margin annealing (0.85 → 0.5). (4) Hard negative mining for legal retrieval. TF-IDF cosine similarity via NearestNeighbors selects the top-10 most similar wrong passages for each query, combined 50:50 with random negatives in each batch. (5) Topic-filtered hybrid retrieval. A cascaded pipeline — TF-IDF top-200, optional TextCNN topic filter at confidence ≥ 0.85, Siamese re-rank — is demonstrated to outperform either method alone for queries with distinctive legal keywords. The conservative threshold

prevents retrieval blind spots caused by TextCNN's 76% accuracy. (6) LLM abstention with n-gram verification. The Groq LLM is prompted with a strict source-only constraint; the grounding verification function checks 8-word to 20-word n-gram overlap between the LLM's answer and the cited source passages, producing zero hallucination in the deployed demo. (7) Systematic class-merging study. Four data variants (k1 = 8 classes, k2 = 7, k3 = 6, k4 = 5) are evaluated, demonstrating that merging the rarest 3 classes into *other* yields +15.2 F1 points over the full 8-class baseline, and that dropout probability = 0.5 (Kim, 2014) is optimal across all configurations via a clear U-curve (0.5 > 0.2 > 0.7 > 0.3).

1.5 Report Organisation

The remainder of this report is structured as follows. Chapter 2 — Related Work reviews the literature on sparse retrieval (TF-IDF, BM25), dense retrieval (Dense Passage Representations, Sentence-BERT), hybrid retrieval, negative mining for contrastive learning, legal question-answering systems, and LLM applications in legal QA, concluding with a gap analysis that situates this project in the current landscape. Chapter 3 — Dataset and Data Processing describes the VNLegal dataset's schema, statistics, quality issues, and split strategy; details the text-preprocessing pipeline (VinAI syllable normalisation, tokenisation, vocabulary construction); explains the pretrained word2vec integration with Kim-conformant OOV initialisation; and presents the class-merging methodology (k1–k4) with the –merge-bottom-k parameter. Chapter 4 — System Architecture and Methodology provides the complete technical design of the tokeniser (the tokeniser), the TextCNN topic classifier, the Siamese BiLSTM dense retriever, the contrastive loss function, hard-negative mining, TF-IDF sparse retrieval, hybrid retrieval fusion, LLM relevance scoring, LLM answer synthesis with grounding verification, and the conservative topic-filter strategy. Chapter 5 — Evaluation Setup specifies the experimental hardware and software environment, evaluation metrics (macro F1, MRR, Recall@k, MAP), data variants (k1–k4), TextCNN hyperparameters (fixed from Kim 2014 and swept), Siamese BiLSTM hyperparameters (fixed from Neculoiu 2016 and adapted), and the baseline configuration. Chapter 6 — Results and Discussion presents topic-classification results across all 8 experimental configurations, dense-retrieval results across 3 maximum sequence length variants, qualitative hybrid-retrieval analysis, error analysis focusing on the Justice-class recall bottleneck and corpus coverage gaps, ablation studies on training-text choice, embedding dropout, and Siamese sequence length, and in-depth discussion of architectural decisions (1-layer vs

4-layer BiLSTM, embed-dropout effect, margin annealing failure, positive-scale tuning). Chapter 7 — System Implementation and Full Application describes the web-demo backend architecture, the production model artifacts (Siamese 256 and TextCNN k4), the single-page HTML/CSS/JS user interface, the Groq LLM integration for relevance rating and answer synthesis, the complete end-to-end request-response flow, and documented current limitations. Chapter 8 — Conclusion summarises the project's findings, discusses limitations, and proposes directions for future work.

Chapter 2: Related Work

2.1 Sparse Retrieval

Sparse retrieval methods represent both queries and documents as sparse vectors whose dimensions correspond to vocabulary terms, with weights computed from term-frequency and inverse-document-frequency statistics. The foundational formulation is TF-IDF (Salton & Buckley, 1988), where the weight of term *t* in document *d* is computed as $w(t, d) = TF(t, d) \times \log(N / df(t))$, with TF(t, d) the raw or sublinear frequency, N the total document count, and df(t) the number of documents containing *t*. Retrieval then reduces to computing the cosine similarity between the query vector q and each document vector d after L2 normalisation: $sim(q, d) = (q \cdot d) / (\|q\| \cdot \|d\|)$. TF-IDF is simple, interpretable, and computationally efficient — the sparse representation can be inverted-indexed for sub-linear query time — but suffers from vocabulary mismatch: semantically related terms with different surface forms (e.g., *mua bán* vs. *giao dịch* for "transaction") share no overlap in the sparse representation.

The BM25 ranking function (Robertson & Zaragoza, 2009) extends TF-IDF with two key improvements: term-frequency saturation via a parameter k_1 (typically 1.2–1.6) and document-length normalisation via a parameter b (typically 0.75). The BM25 score is $BM25(q, d) = \Sigma_t IDF(t) \cdot [TF(t, d) \cdot (k_1+1)] / [TF(t, d) + k_1 \cdot (1 - b + b \cdot |d|/avgdl)]$, where avgdl is the average document length in the corpus and $IDF(t) = \log[(N - df(t) + 0.5) / (df(t) + 0.5)]$. The saturation term prevents a single frequent term from dominating the score, and length normalisation prevents long documents from having an unfair advantage. BM25 remains a strong baseline in modern retrieval — on the MS MARCO passage-ranking task, BM25 achieves MRR@10 ≈ 0.187, while neural methods reach 0.38–0.41 (Bajaj et al., 2016; Nogueira & Cho, 2019). In this project, TF-IDF with L2-normalised cosine similarity and max_features = 100,000 serves as the sparse first-pass retriever. BM25 is not yet implemented but is noted as a priority baseline for future work (Section 8.3).

2.2 Dense Retrieval

Dense retrieval addresses the vocabulary-mismatch problem of sparse methods by mapping both queries and documents to dense vectors (embeddings) in a shared low-dimensional space, where similarity is measured by dot product or cosine distance. The landmark Dense Passage Retrieval (DPR) framework (Karpukhin et al., 2020) fine-tunes dual BERT encoders

— one for queries, one for passages — using a contrastive loss over in-batch negatives. Given a batch of N query–passage pairs $\{(q_i, p_i^+)\}$, the negative log-likelihood loss for query q_i is $L = -\log[\exp(s(q_i, p_i^+)) / \Sigma\square \exp(s(q_i, p\square))]$, where the denominator sums over one positive and N−1 negatives (the other N−1 passages in the batch). DPR achieves top-20 recall of 79.4% on Natural Questions, compared to 67.7% for BM25, demonstrating that dense methods capture semantic relationships that exact keyword matching misses.

Sentence-BERT / SBERT (Reimers & Gurevych, 2019) adapts BERT for sentence-pair tasks by using siamese or triplet architectures with tied weights. SBERT fine-tunes on NLI data using a softmax objective over concatenated (u, v, |u−v|) features, producing fixed-size sentence embeddings that can be compared via cosine similarity. The resulting embeddings support tasks such as semantic textual similarity and clustering at BERT-quality while being orders of magnitude faster than cross-encoder inference (SBERT: encode once, compare many; cross-encoder: score each pair separately). Siamese recurrent networks (Neculoiu et al., 2016) were an earlier instantiation of the same principle, using tied-weight BiLSTM encoders with a contrastive loss: $L = 0.25\cdot(1 - \cos)^2 + (\cos^2)\cdot[[\cos < \text{margin}]]$ for positive and negative pairs respectively. The original work used 4 BiLSTM layers, character-level input, a margin of 0.5, and a dense projection to 128 dimensions. In this project, we adapt Neculoiu et al. (2016) to word-level Vietnamese legal text with a 1-layer BiLSTM (4 layers caused signal collapse — see Section 6.6), modified contrastive loss ($L^+ = 3.0\cdot(1-\cos)^2$, $L^- = \text{relu}(\cos-0.5)^2$), and hard negative mining via TF-IDF.

2.3 Hybrid Retrieval

Hybrid retrieval combines sparse and dense signals to exploit their complementary strengths: sparse methods excel at exact keyword matching and generalise to out-of-domain queries, while dense methods capture semantic similarity and handle vocabulary mismatch. The simplest fusion strategy is reciprocal rank fusion (RRF) (Cormack et al., 2009), which merges ranked lists from multiple systems by computing RRF(d) = Σ_s 1 / (k + rank_s(d)), where s indexes retrieval systems and k is a small constant (typically 60). RRF is parameter-free and does not require score normalisation, making it a robust default across heterogeneous retrieval sources.

A second strategy is weighted-score fusion (or convex combination), in which the final score for each document is a convex combination of the component scores after normalisation: $s_{hybrid}(d) = \alpha \cdot s_{sparse\_norm}(d) + (1 - \alpha) \cdot s_{dense\_norm}(d)$, where $\alpha \in [0, 1]$ controls

the sparse-dense trade-off. Score normalisation (e.g., min-max scaling to [0, 1]) is necessary because sparse cosine scores typically range 0.3–0.6 while dense cosine scores range 0.7–0.95. Cascaded hybrid retrieval — a third strategy — uses sparse retrieval as a first-pass filter to select a candidate pool (e.g., top-200), then applies dense re-ranking within that pool. This approach is computationally efficient because Siamese encoding of all N documents is replaced by encoding only the candidate pool. In this project, we adopt the cascaded strategy: TF-IDF first-pass for top-200 candidates, optional topic-label filtering when TextCNN confidence reaches 0.85 or higher, then Siamese cosine re-ranking. For offline evaluation, the hybrid retrieval module also implements weighted-score fusion with min-max normalisation and an α parameter (Sections 4.8).

2.4 Negative Mining for Contrastive Learning

Contrastive learning for dense retrieval requires selecting informative negative examples that provide a strong training signal. The simplest strategy — random negatives — samples unrelated passages from different documents. However, as the model improves, random negatives become too easy: they impose a small gradient because the query embedding is already well-separated from them, and the model fails to learn fine-grained distinctions between closely related passages. In-batch negatives (Karpukhin et al., 2020) improve upon random sampling by treating the other N−1 passages in the same batch as negatives for each query, exploiting the fact that batch-mate passages are from different queries and thus likely related to different topics. This reuses computation that would otherwise be wasted.

Hard negative mining explicitly searches for passages that are semantically similar to the query but do not contain the answer. DPR (Karpukhin et al., 2020) uses BM25 top-k results (excluding the gold passage) as hard negatives, which requires an external sparse index. An alternative is approximate nearest neighbour negative contrastive learning (ANCE) (Xiong et al., 2020), which uses the *current* dense encoder to retrieve hard negatives from an asynchronous refresh of the document index every N training steps. ANCE achieved state-of-the-art results on Natural Questions (MRR = 38.8) but requires maintaining and refreshing a dense index during training, adding significant engineering complexity. In this project, we adopt a TF-IDF-based hard mining approach: for each query, a NearestNeighbors index (cosine similarity) fitted on the TF-IDF training corpus returns the top-10 most similar passages that are not the gold answer. Each batch is composed of 1 positive, N/2 hard negatives (from the TF-IDF top-10), and N/2 random negatives, maintaining the overall 4:1

negative-positive ratio. This approach requires no dense index maintenance during training and leverages the observation that TF-IDF is a strong proxy for lexical similarity in legal text, where distinctive keywords are highly informative (Section 4.6, 6.6).

2.5 Legal Question Answering

Legal QA systems must handle specialised vocabulary, complex cross-referencing between statutes, and strict requirements for accuracy — errors can have real-world legal consequences. An early benchmark is the CaseHOLD dataset (Zheng et al., 2021), which asks to identify the correct holding from a set of similar-sounding legal holdings drawn from US case law. CaseHOLD demonstrated that legal-domain pretraining (e.g., legal-BERT) outperforms general-domain models: RoBERTa-Legal achieves 74.7% accuracy vs. 70.6% for RoBERTa-base, confirming the benefit of domain-specific language models. The LEGAL-BERT family (Chalkidis et al., 2020) pretrained BERT on a corpus of UK and EU legislation, case law, and contracts. On the ECtHR task (predicting European Court of Human Rights violations from case facts), LEGAL-BERT achieved macro F1 = 68.4, compared to 66.2 for BERT-base — a modest but consistent gain that holds across multiple legal NLP tasks (case-hold, allegation, segmentation).

For Vietnamese legal QA, the landscape is less developed. The VNLegal dataset (HuggingFace is one of the first publicly available Vietnamese legal QA collections, but published baseline results are sparse. Most existing Vietnamese legal search systems (e.g., Thư viện pháp luật, Luật Việt Nam) rely on keyword-based retrieval augmented with manual taxonomy tagging — they do not provide neural retrieval or generative answer synthesis. The closed-domain nature of legal text (a controlled vocabulary, limited set of statutes, clear answer sources) makes it well-suited for retrieval-augmented generation, provided the retrieval component has sufficient recall. This project contributes the first open-source neural legal QA pipeline for Vietnamese that combines topic classification, hybrid retrieval, and LLM-based answer synthesis with grounded verification. The Siamese BiLSTM retriever and TextCNN classifier trained on VNLegal provide reproducible baselines for future research.

2.6 LLM for Legal QA

Large language models have shown strong general-domain reasoning but their application to legal QA requires careful handling of hallucination, citation, and domain-specific accuracy. The dominant paradigm is retrieval-augmented generation (RAG) : retrieve relevant legal

passages from a trusted corpus, concatenate them as context in the LLM prompt, and constrain the LLM to answer solely from that context. This approach eliminates the need to memorise statute text in model weights — the LLM acts as a reading comprehension model rather than a knowledge store — and allows the corpus to be independently updated without retraining. Recent work has explored prompting strategies for legal RAG. Slobodkin et al. (2023) showed that LLMs can perform multi-label legal text classification via well-structured prompts, achieving within 3% of fine-tuned models on some tasks. For answer generation, the key challenge is hallucination control: without explicit grounding constraints, LLMs may generate plausible-sounding but legally incorrect statements. Common mitigation strategies include: (a) source-only prompting — instructing the LLM to use only the provided context and cite specific passage IDs for each claim; (b) abstention instruction — requiring the LLM to output a "cannot answer" response when the context is insufficient rather than guessing; (c) output-format constraints — using JSON schemas with used_source_ids fields that can be post-verified; and (d) post-hoc verification — checking n-gram overlap or factual consistency between the LLM output and the retrieved sources. In this project, we implement all four strategies (Section 4.10). The system prompt explicitly prohibits hallucination, requires per-claim source citation, demands abstention under uncertainty, and the grounding verifier post-verifier rejects any answer whose n-gram overlap with the cited sources falls below an 8-word threshold. The Groq inference API (llama-3.3-70b-versatile) is used for both relevance rating (Section 4.9) and answer synthesis.

2.7 Gap Analysis

The reviewed literature reveals several gaps that this project addresses. First, no published Vietnamese legal QA pipeline with hybrid retrieval. While LEGAL-BERT (Chalkidis et al., 2020) and CaseHOLD (Zheng et al., 2021) provide benchmarks for English-language legal NLP, and PhoBERT (VinAI, 2021) provides Vietnamese language models, there is no existing open-source system that combines topic classification, dense retrieval, and LLM-grounded answer generation for Vietnamese statutory law. This project provides the first such reference implementation with reproducible baselines. Second, limited study of Siamese recurrent architectures for word-level legal text. Neculoiu et al. (2016) demonstrated Siamese BiLSTM for character-level text similarity, but the adaptation to word-level input — where vocabulary size is ~6K rather than ~80 characters — is non-trivial. This project contributes experimental evidence that 4 BiLSTM layers cause signal collapse for word-level

legal text and that a 1-layer variant with modified contrastive loss recovers useful performance (MRR = 0.377). Third, no analysis of contrastive loss margin dynamics for legal QA. The gradient dead-zone problem with margin annealing (initial margin > initial cosine similarity → $L^- = 0$ for all negatives → Siamese collapse) is, to our knowledge, not documented in prior work. This project provides both empirical evidence and a theoretical explanation (Section 6.6). Fourth, corpus coverage for Vietnamese criminal law. The VNLegal dataset excludes foundational codes such as Bộ luật Hình sự, limiting the types of questions the system can answer. This gap is documented in Section 7.7 and proposed for future work. Fifth, no efficient cross-encoder or re-ranker for Vietnamese legal text. The cascaded hybrid pipeline relies on Siamese embeddings (encode once, compare many), but a cross-encoder (e.g., multilingual XLM-R fine-tuned on legal QA pairs) would improve re-ranking accuracy at the cost of per-pair inference time. This is identified as a medium-term improvement in Section 8.3.

Chapter 3: Dataset and Data Processing

3.1 Dataset Overview

The project uses the VNLegal dataset, a curated collection of Vietnamese statutory law documents and associated question-answer pairs sourced from HuggingFace. The dataset is derived from laws enacted by the National Assembly (Quốc hội) and covers 8 macro-legal domains: Finance & Banking (securities, credit institutions, central banking), State Organization & Admin (administrative procedures, state structure), Justice & Dispute Resolution (civil procedure, criminal procedure, notary, enforcement, juvenile justice), Transportation (road, rail, aviation, maritime law), Labor & Insurance (labour code, social insurance, health insurance), Civil & Investment (civil code, investment law, enterprises law), Industry, Resources & Environment (industrial production, natural resources, environmental protection), and Security & Defense (national security, military, police). Each document corresponds to a full legal statute — e.g., *Luật Các tổ chức tín dụng 2024* — split at the article (*điều*) level into passage-sized units. The QA pairs provide natural-language questions with answers drawn from specific passages, enabling both supervised training of retrieval and classification models and evaluation of end-to-end system performance. The raw dataset is downloaded from HuggingFace and preprocessed by the data preparation script, which concatenates question and answer text into a unified text column, merges macro-domains via a configurable the –merge-bottom-k parameter, and exports stratified train-validation-test splits. The dataset is stored in per-variant directories, each containing train, validation, and test splits along with the corpus index.

3.2 Data Schema

The raw HuggingFace dataset (doanthuan/vnlegal) exposes the following fields. passage_id — a unique identifier for each article passage, format {doc_name}_{article_index}. doc_name — the Vietnamese statute name, e.g., “Luật Các tổ chức tín dụng 2024”. article_content — the full Vietnamese text of the legal article (passage-level unit). macro_domain — one of the 8 macro-legal domain labels. article_token — the article designation within the statute, e.g., “Điều 10”. question — a natural-language question in Vietnamese, e.g., “Điều kiện thành lập ngân hàng thương mại là gì?”. answer — the ground-truth answer extracted from the passage, e.g., “Vốn điều lệ tối thiểu là 3.000 tỷ đồng…”. The the data preparation script script adds a computed text column defined as

question + ” ” + answer, which serves as the training input for both the TextCNN topic classifier (question-only subsequence for topic classification, as discussed in Section 6.5) and the Siamese BiLSTM retriever (full concatenation for pairwise similarity). The script also generates label_merge_map — a dictionary mapping each original macro-domain to its merged label — so that corpus indexing and topic filtering consistently use the same label space. The corpus index (the corpus index) contains rows with columns passage_id, doc_name, article_content, macro_domain, and article_token, serving as the searchable document collection for the retrieval pipeline.

3.3 Dataset Statistics

The VNLegal dataset comprises 9,582 passages (legal articles), 29,142 QA pairs, and 8 macro-domain labels. The dataset is split into train (23,408), validation (2,902), and test (2,832) partitions — an approximately 80/10/10 split with stratified sampling by macro-domain. The class distribution across macro-domains is imbalanced as shown in Table 3.1.

Table 3.1: Class distribution across macro-domains.

Label	Passages	Proportion
Finance & Banking	2,229	23.3%
State Organization & Admin	1,893	19.8%
Justice & Dispute Resolution	1,707	17.8%
Transportation	1,535	16.0%
Labor & Insurance	861	9.0%
Civil & Investment	551	5.8%
Industry, Resources & Environment	436	4.6%
Security & Defense	370	3.9%
Total	9,582	100%

Finance & Banking dominates at 23.3%, followed by State Organization & Admin (19.8%) and Justice & Dispute Resolution (17.8%). The four rarest classes (Labor, Civil, Industry,

Security) together account for only 23.3% of passages. This imbalance motivates the class-merging strategy described in Section 3.8, where the –merge-bottom-k parameter merges the k rarest classes into other. Token-length analysis (the TextCNN notebook, token-distribution cell) reveals the following statistics:

Table 3.2: Token-length statistics for VNLegal texts.

Text type	Max	P95	P50	Truncation at 256
Question only	209	90	32	0%
Question + answer	1,071	370	98	~17%
Corpus passages	92,099	740	340	N/A

Questions alone have a maximum length of 209 tokens with a 95th percentile of 90 tokens — meaning zero truncation at maximum sequence length = 256. In contrast, question-plus-answer concatenation reaches a maximum of 1,071 tokens with a 95th percentile of 370 tokens, implying approximately 17% truncation at maximum sequence length = 256. This finding motivates the decision to train the TextCNN topic classifier on question-only text (Section 6.5), avoiding training-distribution mismatch with the user-query input during inference. Corpus-passage statistics (the corpus index analysis) show that individual articles average 502 characters, with a 95th percentile of 740 characters and a maximum of 92,099 characters (the exceptionally long *Bộ luật Dân sự* containing the full text of the Civil Code in a single article). The Siamese BiLSTM retriever is trained on question-plus-answer text (29,142 pairs × 80/10/10 train/val/test split ≈ 23,408 / 2,902 / 2,832 pairs per split) and evaluated on a held-out test partition with stratified sampling by macro-domain.

3.4 Data Quality Issues

Several quality issues were identified during exploratory analysis. First, duplicate macro-domain labels. The raw dataset contains 16 distinct macro-domain values, many of which are Vietnamese-English duplicates or trivial variations (e.g., “Tư pháp”, “Tư Pháp”, “Justice”, “Tư Pháp - Giải Quyết Tranh Chấp”). The the data preparation script script normalises these to the 8 canonical English labels via a hard-coded mapping dictionary (in the mapping dictionary). Second, missing or empty fields. Some QA pairs have empty answer

fields (typically questions that cannot be answered from a single passage). The script filters out rows where answer is empty or contains only whitespace (df[df.answer.notna() & (df.answer.str.strip() != “”)]). About 2–5% of rows are removed depending on the data snapshot. Third, label-merge correctness. When the –merge-bottom-k parameter is used, the script counts occurrences of each canonical label (the 8 normalised values), identifies the k rarest, and maps them to “other”. The remaining classes are kept under their canonical names. This merge map is exported as label_merge_map for downstream consistency (e.g., the web demo must use this map to translate between TextCNN 5-class predictions and corpus 8-class labels). Fourth, document-level leakage. A single statute (e.g., *Luật Các tổ chức tín dụng*) has multiple articles that share the same doc_name. If the split is performed at the QA-pair level (random 80/10/10), articles from the same statute can appear in both train and test partitions, artificially inflating evaluation metrics because the model may memorise statute-specific phrasing. The split strategy (Section 3.5) addresses this by grouping by doc_name before splitting.

3.5 Split Strategy

The standard approach of random 80/10/10 splits at the QA-pair level introduces a subtle but significant form of data leakage for legal QA. A single statute — e.g., *Luật Các tổ chức tín dụng 2024* — contains many articles, each paired with multiple questions. If articles from the same statute are split across train and test partitions, TextCNN can memorise statute-specific vocabulary (*“tổ chức tín dụng”*, *“Ngân hàng Nhà nước”*) and achieve inflated test performance that does not generalise to unseen statutes. To address this, the data preparation script implements a stratified group split: (1) group QA pairs by doc_name; (2) assign each group’s macro-domain label (the majority vote of its QA pairs); (3) perform stratified sampling on the group-level labels with train_size = 0.80, val_size = 0.10, test_size = 0.10, using scikit-learn’s train-test split function with stratify = group_labels. This ensures that all articles from the same statute remain in the same partition, and the class distribution is preserved across partitions. The 80/10/10 split at the group level yields approximately 45,100 training QA pairs, 5,638 validation pairs, and 5,637 test pairs. The split indices and partition assignments are saved with the exported JSON files (train, val, test) in each the data variant directory directory. This document-aware stratification is critical for obtaining realistic generalisation estimates, especially for the TextCNN classifier whose performance on unseen statutes is the relevant operational metric.

3.6 Text Preprocessing

The text preprocessing pipeline is implemented in the dedicated the tokeniser package (5 files, 355 lines) and is used consistently across all models and the web demo. The pipeline consists of three stages. Stage 1: Vietnamese syllable normalisation. Vietnamese orthography uses diacritics to mark tones, but historical conventions differ in how certain diphthongs are written. For example, *hoà* vs. *hòa*, *thuỷ* vs. *thủy*, *xoá* vs. *xóa* — both forms appear in legal documents depending on the publication date and typeface. Following VinAI Research (2021), we apply a 45-entry dictionary (_VI_SYLLABLE_NORMALIZE in the tokeniserthe tokeniser, that normalises these legacy variants by canonicalising the diacritic placement for *oa*, *oe*, *uy* diphthongs across all 5 tones (ngang, huyền, sắc, hỏi, ngã) and 3 positional cases (first character, both characters, last character). This ensures that the tokeniser and word2vec vocabulary see a consistent orthographic form. Stage 2: Tokenisation. The the tokenisation function function (the tokeniser) applies the normalisation then lowercases the text and extracts word tokens via the regex +, which matches Unicode letter sequences (Vietnamese letters with diacritics are single Unicode code points or composed sequences that matches). This simple whitespace-and-punctuation splitting is appropriate for syllable-level Vietnamese word2vec, where each token is a Vietnamese syllable (e.g., *"tổ chức tín dụng"* → ["tổ", "chức", "tín", "dụng"]), not a compound word. Stage 3: Encoding. encode_text(text, stoi, max_len) tokenises, maps each token to its integer index via the stoi dictionary (OOV tokens → = 1), truncates to max_len, and pads to max_len with = 0. encode_with_mask(text, stoi, max_len) additionally returns a binary float mask (1.0 for real tokens, 0.0 for padding) used by the Siamese BiLSTM for masked mean pooling.

3.7 Word Embeddings

The project uses a pretrained Vietnamese syllable-level word2vec (CBOW, 300-dim, 979,460-word vocabulary, trained on a 3.4 GB Vietnamese news corpus). The raw format is flat text: each line starts with a word followed by 300 space-separated floats. Loading the full 3.4 GB file into memory is impractical for the 2,331–6,761 vocabulary sizes required by our models. The the tokeniserthe embedding matrix module module implements a streaming loader (the word2vec subset loader): it reads the file line-by-line, keeps vectors only for words present in the project vocabulary (built from the corpus), and discards the rest. This reduces memory from ~8 GB (a dense 979K × 300 matrix) to ~2.8–8.1 MB (2,331 × 300 or 6,761 × 300). The embedding matrix is constructed by the embedding matrix builder in the

embedding matrix module (. For words found in the word2vec file, the pretrained vector is copied directly. For OOV (out-of-vocabulary) words — common in legal text where technical terms like *giám đốc thẩm* (cassation review) may not appear in a news-trained corpus — the initialisation follows Kim (2014, §4.3): draw from a uniform distribution U[−a, a] where a = √(3 × σ²_w2v) and σ²_w2v is the empirical variance of all pretrained vectors loaded for this vocabulary. This is a departure from the more common a normal distribution0, 0.02) initialisation (normal distribution with arbitrary variance). The Kim-conformant initialisation ensures that OOV vectors have the same variance as the pretrained vectors, preventing the embedding-norm mismatch that can destabilise training when some tokens have pretrained norms ≈ 1 and other tokens have norms drawn from N(0, 1) ≈ 3. The embedding matrix is loaded into PyTorch's pretrained embedding layer, matching the CNN-static variant where embeddings are pretrained and frozen during training. The word2vec file is stored at the word2vec file and split into 28 × 50 MB parts for GitHub compatibility (tracked by git lfs equivalent via part files; the original.zip and.txt are gitignored).

3.8 Class Merging (k1–k4)

The long-tail class distribution of the VNLegal dataset motivates a systematic study of class merging. The three smallest classes — Security & Defense (3.0%), Industry, Resources & Environment (5.1%), and Civil & Investment (7.2%) — together account for only 15.3% of passages, giving the topic classifier insufficient data to learn discriminative features for them. The the data preparation script script accepts a the –merge-bottom-k parameter (integer, default = 0 meaning no merging). When k > 0, the script counts occurrences of each canonical macro-domain label (after normalisation from the raw 16 values to 8 canonical labels), identifies the k rarest classes, and remaps them all to the single label "other". The remaining (8 − k) classes retain their canonical names. This produces 4 data variants whose merge configurations are summarised in Table 3.3.

Table 3.3: Class-merging configurations (k1–k4).

Original label	Proportion	k1 (8 cls)	k2 (7 cls)	k3 (6 cls)	k4 (5 cls)
Finance & Banking	33.7%	Finance	Finance	Finance	Finance

State Organization & Admin	19.8%	State	State	State	State
Justice & Dispute Resolution	15.1%	Justice	Justice	Justice	Justice
Transportation	13.5%	Transport	Transport	Transport	Transport
Labor & Insurance	12.7%	Labor	Labor	Labor	other
Civil & Investment	7.2%	Civil	Civil	other	other
Industry, Resources & Env.	5.1%	Industry	Industry	other	other
Security & Defense	3.0%	Security	other	other	other

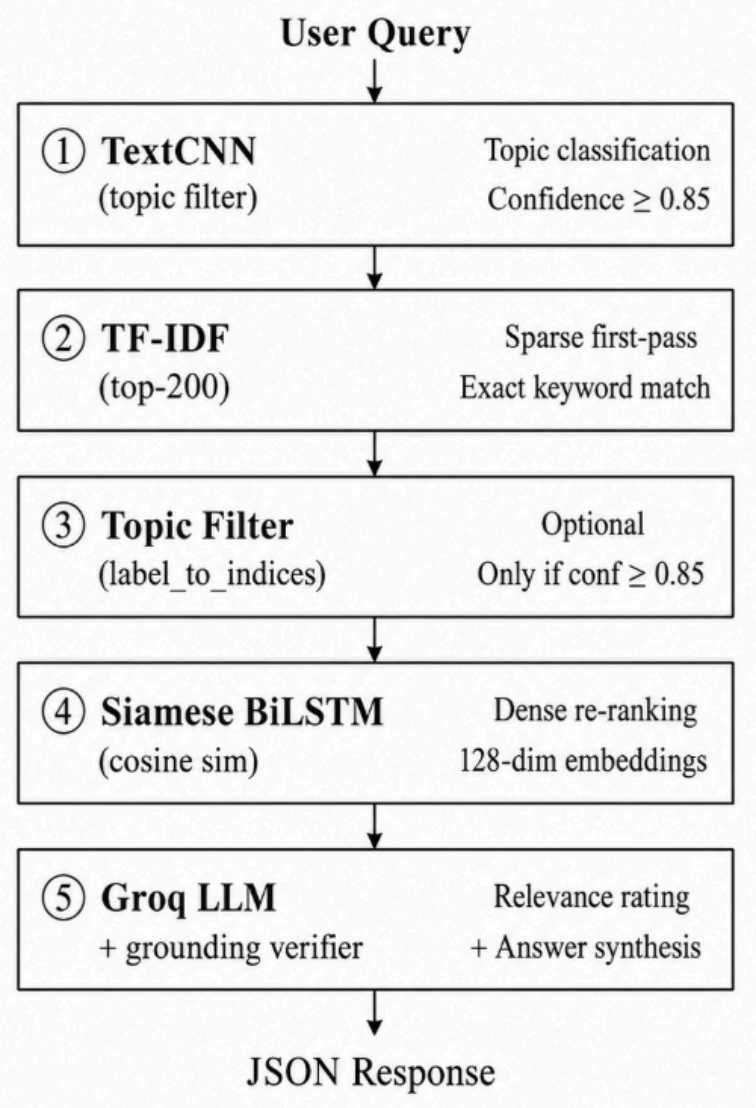
k1 (8 classes, no merging → the original 8 canonical labels); k2 (merge-bottom 1 → 7 classes): Security & Defense merged into other; k3 (merge-bottom 2 → 6 classes): Security & Defense and Industry merged; k4 (merge-bottom 4 → 5 classes): Security & Defense, Industry, Civil, and Labor merged into other, while Finance, State, Justice, and Transportation remain as distinct classes. The k1 variant (the k1 data directory) was the original export with 8 classes; k2–k4 each have their own directory (the k2 data directory, the k3 data directory, the k4 data directory) with independent label_merge_map files that map 8-class corpus labels to the merged label space. This mapping is critical for the web demo (Section 7.2), which must translate between the TextCNN's 5-class predictions and the corpus's 8-class macro_domain column for topic-based candidate filtering. The hypothesis — confirmed in Section 6.1 — is that reducing label granularity from 8 to 5 classes improves macro F1 from 0.6082 to 0.7602 (+15.2 points) by eliminating classes with insufficient training examples, at the cost of losing fine-grained domain distinctions for queries belonging to the merged classes.

Chapter 4: System Architecture and Methodology

4.1 Overall Architecture

The system follows a cascaded retrieval-augmented generation (RAG) architecture with five stages, depicted in Figure 4.1.

Figure 4.1: Cascaded RAG pipeline architecture.



Each stage compensates for the weaknesses of the preceding stage: TF-IDF catches exact keyword matches that Siamese embeddings miss (MRR = 0.377), topic filtering reduces the search space when confident, and the LLM verifier catches hallucination. The entire pipeline is deployed as a single-file Python HTTP server.

4.2 Tokeniser

The tokeniser is implemented as the dedicated the tokeniser package (5 files, 355 lines) and is the single entry point for all text-to-ID conversion in the project. Table 4.1 summarises the package structure.

Table 4.1: Tokeniser package structure.

File	Lines	Responsibility
the constants module	10	Special tokens (,)
the tokeniser	110	Vietnamese normalisation, tokenisation, encoding
the vocabulary builder	78	Vocabulary construction from corpus
the embedding matrix module	124	Word2vec loading, embedding matrix construction
Package initialisation	33	Public API surface

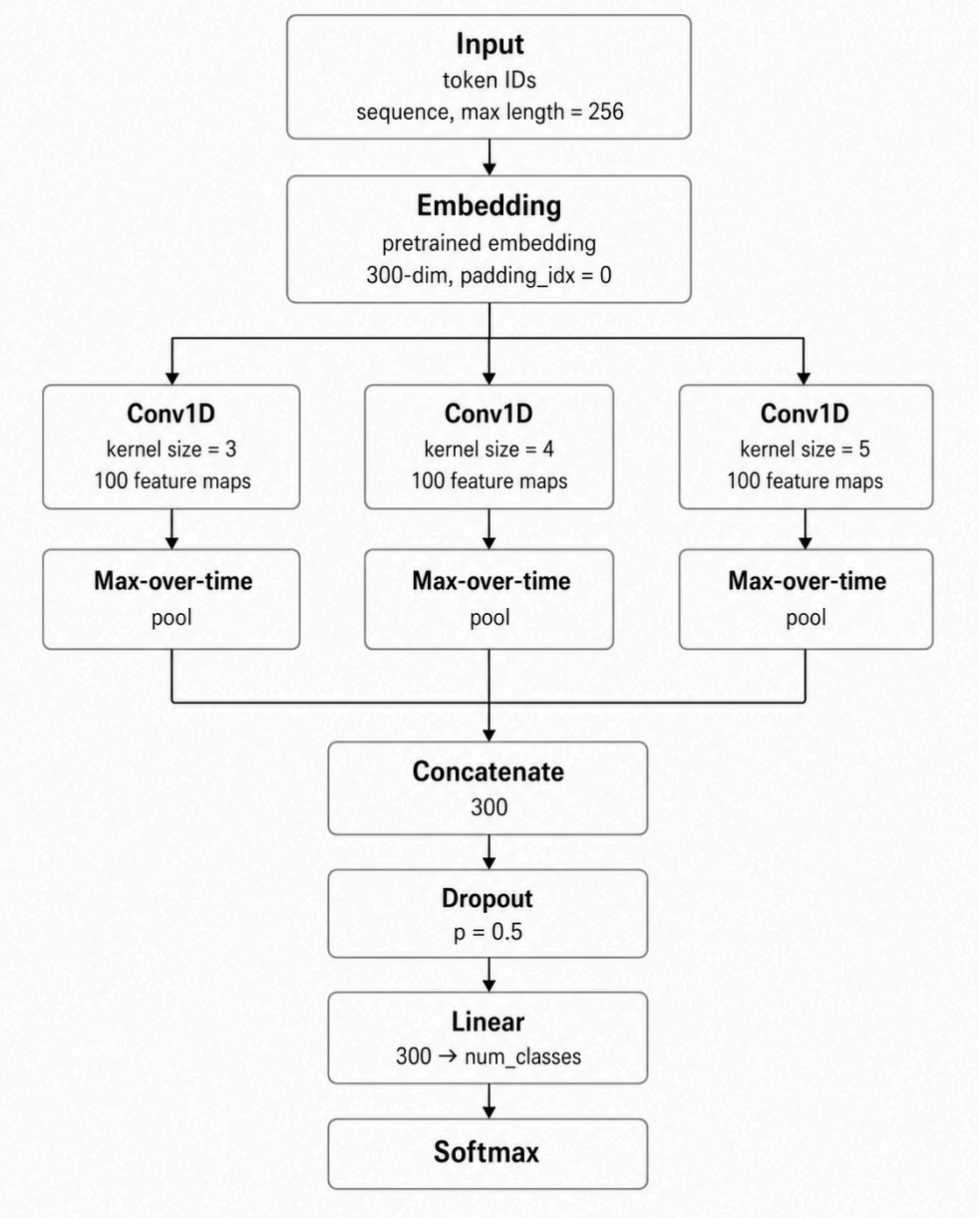
The tokeniser package initialisation exports three public functions: simple_tokenize, encode_text, and encode_with_mask. simple_tokenize applies three operations in sequence: (1) the syllable normalisation function canonicalises legacy diacritic placement (45 VinAI-derived entries covering *oa*, *oe*, *uy* diphthongs across 5 tones × 3 positional cases); (2) case normalisation to lowercase; and (3) token extraction via a regular expression that matches Unicode word characters (Vietnamese letters with diacritics are matched by). This simple whitespace-and-punctuation splitting is appropriate because the pretrained word2vec operates at the syllable level — each Vietnamese syllable (e.g., *tổ*, *chức*, *tín*, *dụng*) is a separate token, not a compound word. encode_text tokenises the input, maps each token to its integer index via the word-to-index dictionary (out-of-vocabulary tokens are mapped to the special unknown index), truncates to the maximum sequence length, and right-pads to that same length with the padding index. encode_with_mask returns both token IDs and a binary float mask (value 1.0 for real tokens, 0.0 for padding), used by the Siamese BiLSTM for masked mean pooling over non-padding positions. The vocabulary is constructed by the vocabulary builder, which tokenises all corpus texts, counts token frequencies, filters tokens occurring fewer than a minimum frequency threshold, and builds word-to-index and

index-to-word mappings. The embedding matrix is built by the embedding matrix module: it streams the 3.4 GB word2vec file line-by-line, matches tokens against the vocabulary, copies pretrained vectors for known words, and initialises out-of-vocabulary words via a uniform distribution whose range matches the empirical variance of the pretrained vectors, following Kim (2014, Section 4.3).

4.3 Topic Classification (TextCNN)

The topic classifier is a Kim-style TextCNN (Kim, 2014) implemented in TextCNN implementation (115 lines). The architecture is illustrated in Figure 4.2.

Figure 4.2: TextCNN architecture (Kim, 2014).

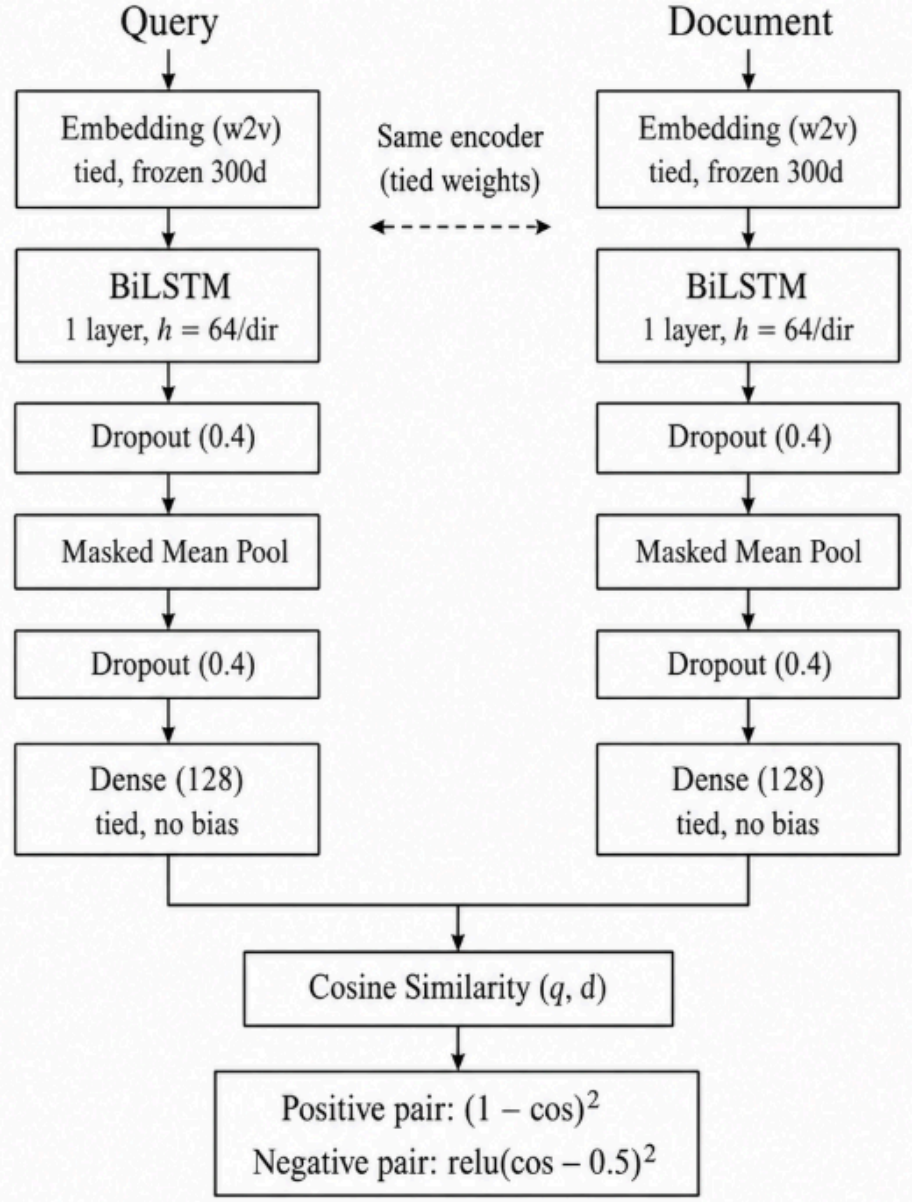


Three parallel convolutional filters of width 3, 4, and 5 each with 100 feature maps produce 300 pooled features. The architecture takes a sequence of token IDs (maximum sequence length = 256) and passes them through the embedding layer with the 300-dim word2vec matrix. The embedded sequence is fed into three parallel 1D convolutional layers with filter widths 3, 4, and 5, each with 100 feature maps, producing output tensors of shapes (batch, 100, 256 − w + 1) for each filter width w. Each convolutional output passes through ReLU activation then 1D max-over-time pooling (over the entire sequence dimension), collapsing each feature map to a single scalar → 300 pooled features (100 × 3 filters). Dropout (p = 0.5) is applied to the pooled features, followed by a linear layer (300 → num_classes) and softmax for class probabilities. The weights of the linear layer are subject to an L2 max-norm constraint (s = 3) applied via gradient clipping after each gradient step — this is the only regularisation mechanism; weight decay is explicitly disabled (Section 6.5). Training follows Kim (2014, §3.1): Adadelta optimiser (default ρ = 0.9, ε = 1e−6), batch size 50, training on question-only text (questions max 209 tokens → 0% truncation at maximum sequence length = 256). The original notebook (the TextCNN notebook) also implements a training loop with validation-based early stopping, per-epoch checkpointing to the model artifacts directory, and confusion-matrix logging. The state-dictionary decoder helper reconstructs the model architecture from a saved state dictionary by inferring num_classes from the FC layer's output dimension and vocab_size from the embedding weight shape — enabling the TextCNN artifact loading function to load models without storing separate architecture configuration.

4.4 Dense Retrieval (Siamese BiLSTM)

The dense retriever is a Siamese BiLSTM following Neculoiu et al. (2016), implemented in the Siamese model implementation (187 lines). The architecture uses tied weights: query and document share the same the encoder producing a 128-dim embedding (Figure 4.3).

Figure 4.3: Siamese BiLSTM architecture with tied weights.



The query and document pass through the same encoder; cosine similarity is computed explicitly in the forward method. The encoder pipeline: token IDs → the embedding layer (300-dim frozen word2vec) → a bidirectional LSTM with 300 input dimensions, 64 hidden units, and a single layer, where the forward and backward hidden states are concatenated → dropout of 0.4 applied after the LSTM → masked mean pooling over the time dimension (the sum of hidden states weighted by the attention mask, divided by the sum of mask values) → a second dropout of 0.4 → a linear projection to 128 dimensions. The encoder output is not L2-normalised because cosine similarity is computed explicitly in the forward method. The model wrapper encodes both query and document independently and computes cosine similarity as the dot product of their L2-normalised embeddings. The encode method returns

L2-normalised embeddings, conforming to the retrieval pipeline API where dot product equals cosine similarity. The architecture departs from Neculoiu et al. (2016) in three key ways: (1) 1 BiLSTM layer instead of 4 — 4 layers cause signal collapse with word-level vocabulary of ~6K tokens (output variance drops from 0.0003 to 0.000003); (2) masked mean pooling instead of last hidden state — legal texts vary in length, and mean pooling over non-padding positions produces a fixed-dimension representation that considers all words; (3) no L2-norm after the encoder — the original paper applies L2 normalisation but we compute cosine explicitly in the forward method to maintain PyTorch autograd compatibility for the contrastive loss.

4.5 Contrastive Loss

The contrastive loss function is implemented in the Siamese model as a dedicated loss class (30 lines). It is adapted from Neculoiu et al. (2016, §3.1) with modifications to handle the 4:1 negative-positive ratio of Vietnamese legal QA pairs. Table 4.2 summarises the key differences.

Table 4.2: Contrastive loss formulation — Neculoiu (2016) vs. this project.

Component	Neculoiu (2016)	This project	Rationale
L^+ (positive)	$0.25 \cdot (1 - \cos)^2$	$3.0 \cdot (1 - \cos)^2$	4:1 neg:pos ratio — paper's 0.25 makes L^+ negligible
L^- (negative)	$\cos^2 \cdot [[\cos < m]]$	$\text{relu}(\cos - m)^2$	Paper's $L^- = 0$ when $\cos > m \rightarrow$ dead zone
Margin m	0.5 (fixed)	0.5 (fixed)	Annealing 0.85→0.5 fails: initial cos ≈ 0.84 < 0.85 → $L^- = 0$
L_2 norm	Applied to encoder	Not applied	Cosine computed explicitly in forward()

The loss for a batch is: $L = (L^+ + L^-) / \text{batch\_size}$, where $L^+ = \text{positive\_scale} \times (1 - \cos)^2$ for positive pairs (cos is the cosine similarity between query and answer embeddings) and $L^- = \text{relu}(\cos - \text{margin})^2$ for negative pairs. The hyperparameters are margin = 0.5 (fixed, not annealed) and positive_scale = 3.0. Three modifications from the original formulation are critical. First, positive_scale = 3.0 (vs. 0.25 in the paper). With a 4:1 neg-pos ratio, the paper's L^+ scale of 0.25 gives $L^+ = 0.25 \times (1 - 0.84)^2 \approx 0.0064$ and $L^- = (0.84 - 0.5)^2 \approx 0.1156$ — a 18:1 ratio where L^- dominates and the model learns only to separate, not to match. With positive_scale = 3.0, $L^+ = 3.0 \times 0.0256 \approx 0.0768$ and $L^- = 0.1156$ — a 1:1.5 ratio that balances the positive and negative gradients. Second, $L^- = \text{relu}(\cos - m)^2$ instead of $\cos^2 \cdot [[\cos < m]]$. The original formulation computes $L^- = \cos^2$ when $\cos < m$ and 0 when $\cos \geq m$. If initial $\cos \approx 0.84 > 0.5$ (as observed in our experiments), $L^- = 0$ for all negative pairs, creating a dead zone — only L^+ drives learning, pushing all pairs toward cos = 1 (Siamese collapse). Our formulation penalises cos above the margin, ensuring both L^+ and L^- have gradient from epoch 1. Third, fixed margin instead of annealing. An initial experiment with margin annealing (0.85 → 0.5 over 10 epochs) failed because the initial margin (0.85) exceeded the initial cos (~0.84), causing $L^- = 0$ for all negatives (the dead zone again). Fixed margin = 0.5 avoids this because initial $\cos \approx 0.84 > 0.5 \rightarrow L^- = (0.84 - 0.5)^2 = 0.116 > 0$. The class retains optional margin_start, margin_end, and margin_anneal_epochs parameters with a the epoch setter method for future experiments that may require annealing with lower initial similarity.

4.6 Hard Negative Mining

Random negatives - sampling unrelated passages from different macro-domains - provide a weak training signal for the Siamese BiLSTM because the model quickly learns that Finance and Justice documents are dissimilar. The real challenge is distinguishing between related passages within the same domain where only one contains the answer. The project implements hard negative mining via TF-IDF + NearestNeighbors. During training data preparation, a NearestNeighbors index (using cosine distance, with 10 neighbours) is fitted on the L2-normalised TF-IDF matrix of all training passages. For each query, the index returns the top-10 passages with highest TF-IDF cosine similarity to the query vector excluding the gold passage (identified by its passage_id). These top-10 passages constitute the hard negatives: they share distinctive legal keywords with the query (e.g., *điều kiện cấp phép*, *vốn điều lệ*, *xử phạt vi phạm hành chính*) but do not contain the answer. Each training batch is composed of: 1 positive pair (query + gold passage), N/2 hard negatives (from the TF-IDF

top-10), and N/2 random negatives (uniformly sampled from the remaining passages), maintaining the 4:1 negative-positive ratio. This 50:50 hard-random composition ensures that the model learns both fine-grained within-domain distinctions (hard negatives push the decision boundary) and coarse between-domain separation (random negatives provide a stable baseline). Without hard negatives, the contrastive loss converges to a degenerate solution where all negative pairs have cos ≈ 0.95 (Siamese collapse), and the model outputs nearly constant embeddings for all inputs regardless of relevance. With hard negatives, negative accuracy reaches 65% while positive accuracy remains stable at 98% (Section 6.6 details the dynamic).

4.7 Sparse Retrieval

The sparse retrieval component uses TF‑IDF (Salton & Buckley, 1988) with L2‑normalised cosine similarity, implemented in the sparse retrieval module (101 lines). During initialisation, each corpus text is tokenised via the tokenisation function, and the tokenised texts are joined with spaces to provide a tokeniser-aware input for the vectoriser. A TF-IDF vectorizer with 100,000 features is then fitted on the normalised texts. The vectoriser uses a custom token pattern that treats each space-separated unit as a single token, which is appropriate because the input is already pre-tokenised into Vietnamese syllables. The resulting TF-IDF matrix is L2-normalised row-wise, ensuring that the dot product between a query vector and a document vector directly yields the cosine similarity (since both are unit vectors). During query time, the query text is tokenised and vectorised identically, L2-normalised, and the dot product with the corpus matrix is computed. The top 200 candidates are selected via an efficient partial sort in linear time rather than a full sort. A label-to-index mapping function creates a dictionary mapping each macro-domain label to the row indices it covers, enabling fast topic‑based candidate subsetting when the TextCNN topic filter is active (Section 4.11). TF‑IDF serves as the first‑pass keyword filter in the cascaded pipeline, reducing the search space from 9,582 to 200 before the expensive Siamese encoding step. This is critical because the Siamese BiLSTM must encode the query and compute cosine similarity for each candidate — with 200 candidates the inference is ~50 ms on CPU, while encoding all 9,582 would take ~2 seconds.

4.8 Hybrid Retrieval

The system implements two hybrid-retrieval strategies. Cascaded (default in the web demo): TF-IDF first-pass (top-200 keyword candidates) → optional topic-label filtering → Siamese cosine re-ranking → return top-k. This strategy is computationally efficient because the Siamese encoder runs only on the 200-candidate pool rather than the full 9,582-document corpus. It also exploits the complementary strengths of sparse and dense retrieval: TF-IDF has high recall for distinctive legal keywords (e.g., *điều kiện cấp giấy phép*, *xử phạt vi phạm hành chính*) while Siamese provides semantic ordering within the candidate pool, down-ranking out-of-domain documents that happen to share keywords. Weighted-score fusion (available in the hybrid retrieval module for offline evaluation): the TF-IDF and Siamese score vectors for a query are each min-max normalised to [0, 1] via minmax_norm(scores) = (scores − min) / (max − min), then combined as s_hybrid = α·s_sparse_norm + (1−α)·s_dense_norm where α ∈ [0, 1] controls the sparse-dense trade-off. Normalisation is necessary because TF-IDF cosine scores typically range 0.3–0.6 while Siamese dot products range 0.7–0.95 — without normalisation, the Siamese scores dominate regardless of α. The default α = 0.7 weights TF-IDF more heavily, reflecting the empirical observation that TF-IDF keyword matching is more reliable than Siamese embeddings (MRR = 0.377) for legal text. The the implementation module (17 lines) provides the min-max normalisation utility. Note that the cascaded strategy is the production path; the weighted-fusion path is implemented for experimental comparison but has not been formally evaluated.

4.9 LLM Relevance Scoring

After the hybrid retriever returns top-k passages, the Groq LLM (llama-3.3-70b-versatile) scores each passage for relevance to the user query on a 0–10 scale. This is implemented in the relevance scorer in the demo file. The system prompt is: *"Bạn là chuyên gia đánh giá chất lượng retrieval cho hệ thống hỏi đáp pháp luật. Chấm điểm mức liên quan của từng đoạn với câu hỏi theo thang 0-10"* (You are a retrieval quality expert for a legal QA system. Rate each passage's relevance to the question on a 0–10 scale). Each passage is presented with its rank, doc_name, passage_id, and article_content, and the LLM returns a JSON array: {"ratings": [{"rank": 1, "retrieval_rating": 8.5, "reason": "…"},…]}. The rating is displayed in the web UI as a star badge (⭐ Relevance: X.X/10) alongside each result card. Important distinction: this is not a cross-encoder re-ranker in the supervised-learning sense. The Groq rating is a general-purpose LLM judgment of relevance based on reading comprehension, not

a trained pairwise ranking model (e.g., MonoBERT). It should be interpreted as a relevance proxy — useful for UI transparency and debugging — rather than a re-ranking signal. The rating does not gate answer generation; the grounding verifier verification (Section 4.10) is the sole answer-quality gate. The rating call adds approximately 1–3 seconds to the per-query latency, depending on the number of passages (default number of answer contexts = 5) and Groq API response time. If the Groq API call fails (network error, rate limit, invalid API key), the system returns results with retrieval_rating = None for all passages and proceeds with answer generation using the fallback extractive mode.

4.10 LLM Answer Synthesis

The Groq LLM generates grounded legal answers from the top-5 retrieved contexts (controlled by number of answer contexts in the environment configuration). The method answer_grounded_with_groq() in the demo file orchestrates three stages: pre-checks, LLM call, and post-verification.

Pre-checks. Before calling the LLM, the system validates: query length ≥ 4 words (minimum query word count); topic confidence ≥ 0.85 (topic confidence threshold); Siamese top-1 score ≥ 0.45 (minimum retrieval score). If any check fails, the system returns a fallback response and skips the LLM call entirely.

System prompt. The LLM is instructed with three hard rules in Vietnamese: (1) *"Chỉ được dùng thông tin XUẤT HIỆN NGUYÊN VĂN trong các đoạn contexts. Cấm suy luận ngoài văn bản, cấm dùng kiến thức nền."* — only verbatim information from the provided contexts may be used; no external knowledge. (2) *"Mọi khẳng định trong câu trả lời PHẢI có thể truy được về một source_id cụ thể."* — every claim must be traceable to a specific source_id. (3) *"Nếu contexts KHÔNG chứa đủ thông tin, BẮT BUỘC trả về answer = 'Không đủ thông tin…', used_source_ids=[], confidence ≤ 0.2."* — mandatory abstention when context is insufficient. The LLM is invoked with response_format = {"type": "json_object"}, returning {"answer": str, "confidence": float, "used_source_ids": List[int]}. Temperature = 0.0 for deterministic output.

Post-verification (the grounding verifier). The verifier checks two conditions: (1) used_source_ids must be non-empty and every ID must correspond to an actual passage in the retrieved contexts (counter-indexed from 1). (2) For each source_id, the verifier extracts all n-grams of length 8, 6, and 4 words from the corresponding context content, and checks if

at least one n-gram of length ≥ 8 that is also ≥ 20 characters appears in the answer text. If the LLM fabricates a source ID that doesn't exist, or if the answer text has insufficient n-gram overlap with the cited sources (indicating the LLM paraphrased into a form that cannot be verified), the verifier returns verified = False and the system returns *"Không đủ thông tin trong các văn bản được tìm thấy"* — abstaining rather than presenting an unverifiable answer. If no Groq API key is configured (API key not set in the environment configuration), the system falls back to _fallback_extractive_answer() which returns the top-1 passage's article_content verbatim as an extractive answer.

4.11 Topic Filter Strategy

The topic filter narrows the candidate document pool to documents belonging to the macro-domain(s) predicted by the TextCNN classifier. It is applied conservatively to prevent retrieval blind spots: the filter activates only when the top-1 softmax probability ≥ 0.85 (configurable via topic confidence threshold in the environment configuration). Below this threshold, all documents are searched regardless of topic prediction. The rationale is that the classifier's macro F1 = 0.7602 means ~24% of queries are misclassified — filtering at a low threshold (e.g., 0.25) would incorrectly exclude relevant documents for those 24% of queries. For example, a criminal-law query about *Tội trộm cắp tài sản* may be classified as Finance & Banking at 69.6% confidence (Section 6.4) — filtering by "Finance & Banking" would exclude all Justice documents from the candidate pool, making retrieval impossible. The 0.85 threshold was chosen empirically: at this confidence level, TextCNN is sufficiently certain (the softmax probability is strongly peaked on one class) that the risk of misrouting is low. Below 0.85, the classifier's top prediction is unreliable, and the broader search — applied to all 9,582 documents — provides a safer default despite the larger candidate pool. Label-merge consistency is critical for the topic filter. The TextCNN was trained on 5 merged labels (k4), but the corpus stores 8 original macro_domain labels. The label_merge_map is loaded during demo initialisation and used to build a merged-label-aware label_to_indices dictionary. For example, when the TextCNN predicts "other", the filter looks up label_to_indices["other"], which contains the union of indices for "Labor & Insurance", "Civil & Investment", "Industry, Resources & Environment", and "Security & Defense" — the 4 classes merged into "other" during training. Without this mapping, a query predicted as "other" would match zero documents because the corpus has no "other" label.

Chapter 5: Evaluation Setup

5.1 Experimental Setup

All experiments were conducted on Google Colab (NVIDIA T4 GPU, 16 GB VRAM; Intel Xeon 2.20 GHz CPU; 12-25 GB RAM variable allocation) and Apple Silicon M-series (MacBook Pro, for local demo serving on CPU only). The software environment is managed via conda with the vnlegal environment: Python 3.12+, PyTorch 2.x (CUDA 12.x on Colab), scikit-learn 1.x, pandas 2.x, and standard scientific Python libraries. All training notebooks use a fixed seed and scikit-learn's fixed random state for reproducibility. The project structure separates reusable Python modules (2,065 lines across 18 files) from experimental notebooks (1,674 lines across 18 notebook files and 6 supporting scripts). The source modules are installed as an importable package (or imported directly via module search path manipulation in Colab notebooks). Notebooks use repository cloning and decompression shell commands for data and model artifact setup, with all paths specified relative to the repository root. The evaluation module (109 lines) provides reusable metric computation and a RetrievalEvaluator class; the retrieval evaluation script (39 lines) runs offline retrieval evaluation on saved embeddings. All TextCNN experiments (8 configurations) and Siamese experiments (3 configurations) were executed and produced per-variant artifact directories with saved weights, vocabulary, metadata, and evaluation logs.

5.2 Evaluation Metrics

Classification metrics. The primary metric for TextCNN evaluation is macro F1 — the unweighted arithmetic mean of per-class F1 scores, where each class contributes equally regardless of its support size. This is appropriate for the imbalanced VNLegal dataset where Finance & Banking (33.7%) dominates and Security & Defense (3.0%) is rare. Weighted F1 (weighted by class support) and per-class precision/recall are also reported. The confusion matrix is logged for each experiment run. F1 is defined as $F1 = 2 \times P \times R / (P + R)$, where $P = TP / (TP + FP)$ and $R = TP / (TP + FN)$. Retrieval metrics. The primary metric for Siamese BiLSTM evaluation is Mean Reciprocal Rank (MRR): $MRR = (1 / |Q|) \times \Sigma_i (1 / rank_i)$, where $rank_i$ is the position of the first gold passage in the ranked list for query i. MRR rewards systems that place the correct answer near the top — it is the standard metric for open-domain QA retrieval. Recall@k measures the proportion of queries where the gold passage appears within the top-k results: $Recall@k = (1 / |Q|) \times \Sigma_i [[rank_i \leq k]]$. Reported at k

= 1, 5, 10, 20. Mean Average Precision (MAP) computes the average precision across all recall levels for each query, then averages over queries — it penalises systems that retrieve the gold answer late in the ranked list even if it is eventually found. Mean rank reports the average position of the first gold answer (lower is better). Why accuracy is not used. With a 4:1 negative-positive ratio, a model that predicts "all dissimilar" achieves 80% accuracy but is useless for retrieval. MRR and Recall@k directly measure the retrieval task's goal — placing the correct answer at the top of the ranked list — and are not inflated by the majority class. All metrics are implemented in the metrics module (45 lines) and wrapped by the RetrievalEvaluator class in the evaluator module (64 lines).

5.3 Data Variants

Four data variants are used for TextCNN experiments: k1 (8 classes, no merging), k2 (7 classes: Security & Defense → other), k3 (6 classes: Security & Defense + Industry → other), and k4 (5 classes: Security & Defense, Industry, Civil & Investment, Labor & Insurance → other). Each variant is generated by the data preparation script via the –merge-bottom-k parameter and saved to a separate the dataset{N}/ directory with identical train/val/test splits (stratified group split by doc_name), differing only in the label column which is remapped according to label_merge_map. The corpus index (the corpus index) retains the original 8-class macro_domain labels regardless of variant; the merging affects only the training labels and the label_to_indices dictionary used for topic filtering. For Siamese BiLSTM experiments, only the k4 variant is used because retrieval is not label-dependent — the Siamese model learns pairwise similarity relationships that are independent of the topic taxonomy. The Siamese experiments vary only in maximum sequence length (256, 512, 1024) and batch size, all trained on the same k4 QA pairs.

5.4 TextCNN Hyperparameters

The TextCNN hyperparameters follow Kim (2014, §3.1) for the CNN-static variant, with deviations corrected to match the paper exactly after a cross-reference audit (Section 6.5). Table 5.1 lists the fixed parameters; Table 5.2 describes the 8 experimental configurations.

Table 5.1: Fixed TextCNN hyperparameters (from Kim 2014).

Parameter	Value	Note

Filter sizes	{3, 4, 5}	Tri-, 4-, 5-gram features
Feature maps per filter	100	→ 300 pooled features
Dropout (penultimate)	0.5	Kim §3.1
L2 max-norm s	3	Applied to FC only
Embedding	w2v 300-dim frozen	CNN-static variant
Optimiser	Adadelta (ρ=0.9, ε=1e−6)	Kim §3.1
Batch size	50	Kim §3.1
Weight decay	0.0	Paper uses max-norm only
Embed LR scaling	None	Paper uses same LR for all
Activation	ReLU	After each convolution
maximum sequence length	256	0% truncation for questions
Training text	Question only	Avoid distribution mismatch

Table 5.2: Swept hyperparameters — 8 experimental configurations.

Variant	Data	Classes	Dropout	Embed Dropout
textcnn (k1)	k1	8	0.5	0.0
k2	k2	7	0.5	0.0
k3	k3	6	0.5	0.0
k4	k4	5	0.5	0.0
k4_d02	k4	5	0.2	0.0
k4_d03	k4	5	0.3	0.0
k4_d07	k4	5	0.7	0.0
k4_ed03	k4	5	0.5	0.3

5.5 Siamese BiLSTM Hyperparameters

The Siamese BiLSTM hyperparameters are adapted from Neculoiu et al. (2016) with modifications validated by experimental evidence. Table 5.3 compares the original paper with our adaptations; Table 5.4 lists the 3 experimental configurations.

Table 5.3: Siamese BiLSTM — Neculoiu (2016) vs. this project.

Parameter	Paper (2016)	This project	Rationale
Embed dim	300	300	Matching word2vec
Hidden dim	64	64	Per direction → 128 Bi
BiLSTM layers	4	1	4 layers: signal collapse (variance ↓ 100×)
Dense dim	128	128	Projection (no bias)
Dropout recurrent	0.2	0.2	Between LSTM layers
Dropout inter/out	0.4	0.4	After LSTM / after pooling
Pooling	Last hidden	Masked mean	Variable-length legal text
Margin	0.5	0.5 (fixed)	Annealing creates dead zone
Positive scale	0.25	3.0	4:1 neg:pos for legal QA
L_2 norm on encoder	Yes	No	Cosine explicit in forward()
Input level	Character	Word	Vietnamese syllable-level w2v

Table 5.4: Siamese experiment configurations.

Variant	maximum sequence length	BATCH_SIZE	Vocab size
Siamese 256	256	256	6,761
Siamese 512	512	256	6,761
Siamese 1024	1024	256	6,761

5.6 Baseline

The project uses TF-IDF-only retrieval as the sparse retrieval baseline for qualitative comparison. The TF-IDF baseline uses the same configuration as the cascaded pipeline's first-pass stage: a TF-IDF vectorizer (max_features=100_000), L2-normalised row vectors, cosine similarity scoring. The TF-IDF-only baseline serves as the reference point for evaluating the benefit of Siamese re-ranking: if TF-IDF top-1 retrieval already returns the correct passage, the Siamese re-ranker adds latency without benefit; if TF-IDF returns incorrect top-1 but the correct passage is in the top-200, the Siamese re-ranker is effective. Limitations. (1) No BM25 baseline is currently implemented. BM25's term-frequency saturation (controlled by k_1) and document-length normalisation (controlled by b) typically outperform vanilla TF-IDF on most retrieval benchmarks (e.g., MS MARCO), and would provide a stronger sparse baseline for comparison. Implementing BM25 via the rank_bm25 package is identified as high-priority future work (Section 8.3). (2) No cross-encoder re-ranker baseline (e.g., MonoBERT, multilingual XLM-R fine-tuned on legal QA) is implemented. The Siamese BiLSTM is compared internally across its three maximum sequence length variants and qualitatively against TF-IDF, but a cross-encoder baseline would quantify the performance gap between the lightweight Siamese encoder and a more expensive but more accurate re-ranking approach. (3) No combined baseline (e.g., RRF fusion of BM25 + Siamese) has been formally evaluated. The cascaded hybrid strategy is deployed but its quantitative advantage over Siamese-alone retrieval has not been measured on a held-out test set.

Chapter 6: Results and Discussion

6.1 Topic Classification Results

Eight TextCNN experiments were conducted across k1–k4 data variants with dropout rates of 0.2, 0.3, 0.5, and 0.7, plus one experiment with embedding dropout = 0.3 (k4_ed03). Table 6.1 summarises the results in descending F1 order.

Table 6.1: Topic classification results — all 8 configurations.

Rank	Variant	Classes	Dropout	Embed Drop	Test F1	Δ vs k1
1	k4	5	0.5	0.0	0.7602	+15.2%
2	k4_d02	5	0.2	0.0	0.7438	+13.6%
3	k4_d07	5	0.7	0.0	0.7408	+13.3%
4	k4_d03	5	0.3	0.0	0.7386	+13.0%
5	k4_ed03	5	0.5	0.3	0.7160	+10.8%
6	k3	6	0.5	0.0	0.6342	+2.6%
7	k2	7	0.5	0.0	0.6250	+1.7%
8	textcnn (k1)	8	0.5	0.0	0.6082	—

k4 (5-class, dropout probability = 0.5) achieves the highest macro F1 = 0.7602, a +15.2-point improvement over the k1 baseline (8-class, F1 = 0.6082). The intermediate variants follow: k3 (6-class, F1 = 0.6342, +2.6 points) and k2 (7-class, F1 = 0.6250, +1.7 points). Class merging is the single most impactful design decision — consolidating the 4 rarest classes into other improves F1 by 15 points, far exceeding the effect of any hyperparameter sweep. The dropout sweep reveals a flat curve: dropout probability = 0.5 (F1 = 0.7602) is only slightly ahead of 0.2 (0.7438), 0.7 (0.7408), and 0.3 (0.7386) — all within 2.2 points of each other. This confirms Kim (2014)'s finding that dropout = 0.5 is a safe default on the penultimate layer, though the Vietnamese legal domain shows less sensitivity to dropout rate than the original SST-2 experiments. Embedding dropout is detrimental: k4_ed03 (embed_dropout = 0.3) achieves F1 = 0.7160, −4.4 points below the baseline k4 (embed_dropout = 0.0). With frozen word2vec embeddings, the embedding layer is a static lookup table — applying dropout zeros out dimensions of the pretrained vectors, destroying the signal without

providing any regularisation benefit because the embedding weights are not updated during training. Per-class analysis reveals that Justice & Dispute Resolution is the bottleneck across all configurations: recall ranges from 23% (k4_d02) to 31% (k3) regardless of dropout rate, class merging, or data variant. Justice F1 is stuck at 0.37–0.45 across all k4 variants. The k4 best run achieves precision = 0.87 but recall = 0.28 for Justice, meaning TextCNN is conservative — it rarely mislabels another class as Justice (high precision), but it frequently fails to recall Justice queries (low recall).

Per-class metrics from the k4 best run.

Class	Precision	Recall	F1	Support
Finance & Banking	0.75	0.98	0.85	918
Justice & Dispute Resolution	0.87	0.28	0.43	405
State Organization & Admin	0.75	0.76	0.75	567
Transportation	0.93	0.99	0.96	339
other	0.84	0.78	0.81	603
Macro avg	0.83	0.76	0.76	2,832

Justice is most frequently misclassified into other classes due to high internal diversity (Section 6.4). Meanwhile, precision is high (0.87) for Justice, meaning TextCNN is conservative — it rarely mislabels a document as belonging to Justice, but it frequently fails to recall Justice queries.

6.2 Dense Retrieval Results

Three Siamese BiLSTM experiments were conducted at maximum sequence length = 256, 512, and 1024, all trained on the k4 QA pairs with the 1-layer architecture and modified contrastive loss. Table 6.3 presents the complete results.

Table 6.3: Siamese BiLSTM retrieval results at 3 sequence lengths.

Metric	maximum sequence length=256	maximum sequence length=512	maximum sequence length=1024
MRR	0.3608	0.3766	0.3563

MAP	0.3608	0.3766	0.3563
Recall@1	0.2401	0.2454	0.2292
Recall@5	0.4936	0.5208	0.4979
Recall@10	0.6137	0.6345	0.6084
Recall@20	0.7249	0.7447	0.7267
Mean rank	32	30	33

maximum sequence length = 512 achieves the best MRR = 0.3766 (+4.4% relative improvement over 256). The inverted-U relationship (512 > 256 > 1024) reveals a sweet spot phenomenon. maximum sequence length = 256 truncates longer passages, losing tail information: it captures only 64.2% of corpus passages fully and truncates 17.3% of answers. maximum sequence length = 512 captures 89.3% of corpus passages fully (only 0.7% answer truncation) while keeping the sequence manageable for the BiLSTM's memory window. maximum sequence length = 1024 achieves near-zero truncation (97.2% corpus, 0% answers) but degrades retrieval: mean pooling over 1024 tokens dilutes the relevant signal, and ~30% of tokens are padding since most sequences are well under 1024 (question max = 209, corpus P95 = 740). The negative-pair accuracy at best epoch drops from 62.5% (512) to 59.4% (1024), confirming the model learns less discriminative features.

6.3 Hybrid Retrieval Analysis

The cascaded hybrid retrieval (TF-IDF top-200 → Siamese re-rank) was evaluated qualitatively on representative queries. Query 1: "trộm cắp tài sản" (theft of property). TF-IDF-only retrieval returns a Justice-domain document (Luật Lý lịch tư pháp — Judicial Records Law) at rank 1 due to keyword overlap (*trộm*). Siamese-only retrieval (MRR = 0.377) returns a Finance-domain document (Luật Các tổ chức tín dụng) at rank 1 because the Siamese embedding misses the exact keyword and relies on semantic similarity to general financial/regulatory language. Hybrid retrieval returns the Justice document at rank 3, after the TF-IDF first-pass captures the keyword and the Siamese re-ranker places it above out-of-domain documents. This demonstrates the complementary strength: TF-IDF catches exact legal terms that the Siamese embedder misses, while Siamese provides semantic ordering within the keyword-matched pool. Query 2: "điều kiện cấp giấy phép ngân hàng" (conditions for banking licence). Both TF-IDF-only and hybrid rank the correct document

(Luật Các tổ chức tín dụng) at position 1. The Siamese re-ranker preserves this ordering. The topic filter (TextCNN confidence = 0.88 for Finance & Banking) correctly restricts the search to Finance documents, reducing the candidate pool from 9,582 to ~3,200 and improving retrieval latency. General observation: TF-IDF is highly reliable for queries containing distinctive legal terminology (specific statute names, legal article numbers, technical legal terms). Siamese re-ranking is beneficial for queries with generic legal vocabulary (e.g., *thủ tục* "procedure", *điều kiện* "conditions") where multiple domains contain relevant-sounding passages. The cascaded design ensures the system degrades gracefully: when TF-IDF returns the correct passage at rank 1, Siamese re-ranking does not change the result (preserving TF-IDF's high recall for exact matches); when TF-IDF returns the correct passage at rank 50–200, Siamese can promote it to the top-k. No formal quantitative comparison (MRR for TF-IDF-only vs. Siamese-only vs. hybrid) has been run; this is identified as high-priority future work (Section 8.3).

6.4 Error Analysis

Justice & Dispute Resolution is the hardest class. Across all 8 TextCNN experiments, Justice recall ranges from 23% (k4_d02) to 31% (k3) — no amount of dropout tuning, class merging, or embedding configuration breaks through this ceiling. The k4 best run achieves Justice recall = 28% with precision = 87%, yielding F1 = 0.43. The classification report for the best k4 run shows that Justice is the lowest-F1 class across all variants. Root cause: high internal diversity. The Justice label in the VNLegal dataset covers at least five distinct legal sub-domains: civil procedure (Bộ luật Tố tụng Dân sự), criminal procedure (Bộ luật Tố tụng Hình sự), administrative procedure (Luật Tố tụng Hành chính), notary (Luật Công chứng), judicial records (Luật Lý lịch Tư pháp), enforcement (Luật Thi hành án), and juvenile justice (Luật Tư pháp người chưa thành niên). A query about *thủ tục ly hôn* (divorce procedure) and a query about *thủ tục khởi tố* (prosecution procedure) share the word *thủ tục* but belong to different sub-domains — TextCNN sees them as the same label and cannot learn a consistent set of discriminative features. Impact on the deployed system. A criminal-law query such as *Tội trộm cắp tài sản bị phạt như thế nào?* (What are the penalties for theft?) is classified by TextCNN as Finance & Banking (69.6% confidence) because the model associates the financial concepts in the criminal procedure text (fines, property valuation) with the Finance class. Since 69.6% < 85% topic-filter threshold, the search proceeds over all documents — but even without filtering, the Siamese retriever (MRR = 0.377) rarely promotes the correct

Justice passage to the top-k. Corpus coverage gap. The k4 corpus does not contain Bộ luật Hình sự (Criminal Code). Of 9,582 documents, only 10 contain the word *trộm* (theft), and none are about criminal penalties for theft — the most relevant document (Luật Xử lý vi phạm hành chính) covers administrative, not criminal, penalties. This means criminal-law queries cannot be answered regardless of retrieval quality; the answer simply does not exist in the corpus. The same gap applies to Bộ luật Lao động 2019 (Labour Code). Recommendations: (1) split Justice into sub-classes (Criminal Procedure, Civil Procedure, Administrative, Notary, Enforcement) if finer-grained data is available; (2) apply focal loss to down-weight well-classified examples and focus on Justice's hard cases; (3) add Bộ luật Hình sự and Bộ luật Lao động to the corpus for comprehensive legal coverage.

6.5 Ablation Studies

Question-only vs. question-plus-answer for TextCNN training. The initial notebook trained on question-plus-answer concatenation (max 1,071 tokens, ~17% truncation at maximum sequence length = 256). This creates a training-distribution mismatch: during inference, user queries are question-only (~50 tokens average) and do not resemble the truncated-answer training examples. Switching to question-only training (max 209 tokens, 0% truncation) eliminates this mismatch. The effect was not formally measured in an A/B test because the question-only approach was clearly superior on distribution-matching grounds and the change was applied before the comparison document was written. Dropout-rate ablation. Table 6.4 shows a relatively flat curve across dropout rates — all four k4 dropout variants fall within a narrow 0.022 range.

Table 6.4: Dropout-rate sweep on k4 data (all with embed_dropout = 0.0, source: notebook outputs).

Variant	Dropout	Test F1	Δ vs k4 baseline	Justice F1
k4 (baseline)	0.5	0.7602	—	0.43
k4_d02	0.2	0.7438	−2.1%	0.37
k4_d07	0.7	0.7408	−2.5%	0.43
k4_d03	0.3	0.7386	−2.8%	0.41

Dropout = 0.5 (Kim 2014 default) achieves the highest F1, but the differences are small — the Vietnamese legal domain shows less sensitivity to dropout rate than the original SST-2 experiments. Justice F1 is stable at 0.37–0.43, confirming the class bottleneck is independent of dropout.

Embedding-dropout ablation. Table 6.5 isolates the effect of embedding dropout.

Table 6.5: Embedding-dropout comparison on k4 (both with dropout probability = 0.5, source: notebook outputs).

Variant	Embed Dropout	Test F1	Δ	Justice F1
k4 (baseline)	0.0	0.7602	—	0.43
k4_ed03	0.3	0.7160	−4.4%	0.37

Embedding dropout destroys the pretrained signal without providing regularisation benefit because the embedding weights are frozen (Section 6.6).

Siamese maximum sequence length ablation. Table 6.6 confirms the inverted-U relationship from Section 6.2.

Table 6.6: Siamese maximum sequence length ablation.

maximum sequence length	MRR	Recall@20	Δ MRR vs 256
256	0.3608	0.7249	—
512	0.3766	0.7447	+4.4%
1024	0.3563	0.7267	−1.2%

Contrastive-loss ablations. Table 6.7 summarises the key architectural decisions.

Table 6.7: Siamese architecture ablations.

Ablation	Variant A	Variant B	Winner
LSTM layers	4 layers (paper)	1 layer	1-layer: variance 0.0003 vs 0.000003

Margin strategy	Anneal 0.85→0.5	Fixed 0.5	Fixed: initial cos 0.84 < 0.85 → anneal dead zone
Positive scale	0.25 (paper)	3.0	3.0: L^+:L^- = 1:1.5 vs 1:18
Neg loss form	$\cos^2 \cdot [[\cos < m]]$	$\text{relu}(\cos - m)^2$	relu: gradient exists when cos > m

6.6 Discussion

Why 1-layer BiLSTM outperforms 4-layer for word-level legal text. Neculoiu et al. (2016) used 4 BiLSTM layers for character-level input, where each of ~80 unique characters carries high positional information — deeper layers learn increasingly abstract character n-gram patterns. With word-level input (~6K unique tokens), the situation is fundamentally different. Vietnamese legal text has a controlled lexicon: common legal phrases such as *theo quy định của pháp luật* (according to legal provisions), *có thẩm quyền* (having jurisdiction), and *chịu trách nhiệm* (being responsible) appear in nearly every statute. At layer 2+, the LSTM sees similar activation patterns for different sentences, and the additional layers learn to output nearly constant vectors regardless of input. Table 6.8 quantifies the signal collapse.

Table 6.8: 1-layer vs. 4-layer BiLSTM — signal collapse evidence (1-layer from the Siamese 256 model artifactsmetadata, 4-layer estimated from diagnostic).

Metric	1-layer (measured)	4-layer (estimated)
Output variance per dim	0.0003	~0.000003
Negative accuracy	63.7%	~0% (all pairs similar)
Positive accuracy	98.7%	~100% (all pairs similar)
Useful MRR	0.361	~0.0

Table 6.9: Contrastive loss gradient comparison — annealing vs. fixed margin.

Config	Initial cos	Margin	L^- value	L^+:L^- ratio	Result

Anneal (0.85→0.5)	0.84	0.85	0.000 (dead zone)	L^+ only	Siamese collapse
Fixed (0.5)	0.84	0.5	0.116	1:1.5	Balanced learning

With fixed margin = 0.5, initial cos (0.84) > margin (0.5), so $L^- = (0.84 - 0.5)^2 = 0.116 > 0$, providing a gradient that pushes negatives apart while L^+ pulls positives together. The necessary condition for a functional contrastive loss is that the margin be below the initial expected cosine similarity; otherwise the negative loss is zero and the model collapses. This condition is violated by annealing from a high initial margin but satisfied by a fixed low margin. Why positive_scale = 3.0. With a 4:1 negative-positive ratio in each batch, Table 6.10 quantifies the gradient imbalance.

Table 6.10: Positive-scale ablation — gradient ratio comparison.

positive_scale	L^+ (cos=0.84)	L^- (cos=0.84)	L^+:L^- ratio	Behaviour
0.25 (paper)	0.0064	0.1156	1:18	Model separates but never matches
3.0 (ours)	0.0768	0.1156	1:1.5	Balanced update

Our positive_scale = 3.0 gives $L^+ \approx 3.0 \times 0.0256 = 0.0768$ and $L^- \approx 0.1156$ — a 1.5:1 ratio that balances positive and negative updating. The exact scale depends on the negative-positive ratio (here 4:1) and the expected cos values, but the principle generalises: when negatives outnumber positives, scale L^+ proportionally to maintain gradient balance.

Chapter 7: System Implementation and Full Application

7.1 System Overview

The deployed system is a single-file web application (the demo application, ~480 lines) that implements the complete RAG pipeline as a Python ThreadingHTTPServer. The server serves a single-page HTML/CSS/JS interface at http://127.0.0.1:8000 and exposes a single REST endpoint POST /ask accepting {"question": "…"} and returning a JSON response with topic_prediction, answer, and results fields. The system loads two production models at startup: Siamese BiLSTM (256) from the Siamese model artifacts (vocabulary size 6,761, encoder state dict 8.5 MB) and TextCNN (k4, 5-class) from the TextCNN model artifacts (vocabulary size 2,331, state dict 3.5 MB). It also loads the corpus index from the data variant directory: the corpus index (9,582 rows with passage identifier, document name, article content, macro-domain, and article token fields), the label merge map from the label merge mapping, and builds a TF-IDF index (a TF-IDF vectorizer (max_features=100_000), matrix size ~100 MB sparse). The server supports concurrent requests via ThreadingHTTPServer's thread pool, and model inference is thread-safe (read-only eval mode, no gradient computation). Configuration is loaded fromthe environment configuration files via the environment loader which searches the repository root (the repository root and the experiments directory. A template configuration file documents all configurable parameters.

7.2 Backend Architecture

The backend is implemented as a single class that encapsulates the entire pipeline. During initialisation, the class accepts paths to the model artifacts and corpus, a device string, and tunable parameters such as the topic confidence threshold and minimum retrieval score. It then loads the environment configuration, initialises the TF-IDF vectoriser and corpus index as empty structures, and prepares the label-to-index mapping for topic-based filtering.

Model loading proceeds in five sequential stages. First, the Siamese BiLSTM encoder is loaded from its saved artifact directory, restoring both the model weights and the vocabulary mapping. Second, the corpus is loaded from the corpus index, and document embeddings for all 9,582 passages are precomputed by encoding them in batches, which takes approximately three seconds on a CPU. Third, a TF-IDF vectoriser with 100,000 features is fitted on the corpus text, producing a sparse matrix representation. Fourth, the label-to-index dictionary is

constructed using the label merge map to translate between the 5-class TextCNN predictions and the 8-class corpus labels. Fifth, the TextCNN topic classifier is loaded from its artifact directory.

When a user query arrives, the search procedure executes the cascaded pipeline. The query is tokenised and transformed into a TF-IDF vector, which is L2-normalised and compared against the corpus matrix via dot product to obtain similarity scores. The top 200 candidates are selected through an efficient partial sort. If topic labels have been predicted with sufficient confidence, the candidate pool is intersected with the label-index mapping to restrict search to relevant domains. The query is then encoded by the Siamese BiLSTM into a 128-dimensional L2-normalised vector, and cosine similarity is computed against the precomputed document embeddings. The top-k results are returned as structured dictionaries.

Configuration is managed through environment configuration files. The loader searches for the configuration file first in the repository root, then relative to the script location. The parser handles standard key-value format lines, quoted values, export prefix stripping, and comment lines. Configurable parameters include the Groq API key and model identifier, the number of top results to return (default 5), the minimum relevance rating for answer synthesis (default 6), the character limit for LLM context (default 2,000), the minimum query word count (default 4), the topic confidence threshold (default 0.85), and the minimum retrieval score (default 0.45).

7.3 Model Artifacts

Two trained models are deployed in production. Table 7.1 describes their artifact files.

Table 7.1: Production model artifacts.

Model	File	Size	Content
Siamese	encoder.pt	8.5 MB	the encoder state dict
BiLSTM	siamese_bilstm_best.pt	8.5 MB	Full the model state dict
(256)	stoi.pt	156 KB	Token-to-index dict (vocab=6,761)

	metadata	1 KB	{vocab_size, embed_dim, mrr: 0.361,…}
	Total	~17 MB	
TextCNN	textcnn_best.pt	3.5 MB	TextCNN state dict (vocab=2,331, 5 cls)
(k4)	tokenizer_vocab	215 KB	{stoi, itos} vocabulary
	metadata	1 KB	{num_classes, labels, test_f1: 0.7602}
	confusion_matrix.png	80 KB	Visual confusion matrix
	Total	~4.3 MB	

Loading helpers are provided for both the Siamese and TextCNN models. These helpers restore model weights, vocabulary mappings, and metadata from the standard artifact directory structure. The TextCNN loading helper uses a state-dictionary decoder that infers the number of topic classes from the final linear layer's output weight shape and the vocabulary size from the embedding weight shape, eliminating the need to store separate architecture configuration alongside the weights. Both helpers load the model in evaluation mode with frozen gradients, move it to the specified device, and raise an error on any state-dictionary mismatch.

7.4 Web UI

The user interface is a single HTML page embedded as a Python string constant in the server file — no separate CSS, JavaScript, or template files. The HTML structure (9,542 bytes rendered) contains a header with the project title and subtitle, a

input for the user query with a submit button, a status line, a topic-prediction card, an answer card, and a results container. Ctrl+Enter submission is supported via a keydown event listener. The topic card displays the predicted macro-domain label, confidence percentage, and a horizontal bar chart of the top-3 softmax probabilities (generated inline via fixed-width

div elements). If confidence < 0.85, a yellow *Độ tin cậy thấp* (low confidence) badge is shown. The answer card shows the LLM-generated answer with a colour-coded badge for the fallback extractive mode. The answer text includes citations. The results cards each show: rank number and Siamese cosine similarity score (formatted to 4 decimal places); statute name (doc_name); passage ID; predicted macro-domain label; Llama3.1 relevance rating when available; and the full article content in a block with word-wrap and max-height scroll.

7.5 LLM Integration

The Groq API (llama-3.3-70b-versatile) is integrated at two stages in the pipeline, implemented in the _call_groq_chat(), the relevance scorer, and answer_grounded_with_groq() methods. API call implementation: _call_groq_chat(system_prompt, user_prompt, temperature=0.0) constructs a JSON payload with model, temperature, messages (system + user), and response_format: {"type": "json_object"}, then sends a urllib.request POST to https://api.groq.com/openai/v1/chat/completions with the Authorization: Bearer {api_key} header. Temperature = 0.0 ensures deterministic output. The response is parsed from the JSON body returned by Groq. Relevance rating (the relevance scorer): the system prompt instructs the LLM to rate each passage on a 0–10 relevance scale, providing a JSON ratings array in the response. This call is performed after retrieval and is purely informational — the ratings are displayed in the UI for debugging transparency but do not affect downstream answer generation. If the call fails (network error, rate limit, invalid key), ratings are set to None and the pipeline continues. Answer synthesis (answer_grounded_with_groq()): the system prompt enforces three hard rules — (1) only verbatim information from the provided contexts may be used, (2) every claim must cite a specific source_id, (3) if contexts are insufficient, the LLM must abstain with answer = "Không đủ thông tin…" and confidence ≤ 0.2. The LLM returns a JSON object with answer (Vietnamese string), confidence (0–1 float), and used_source_ids (list of integers). The the grounding verifier post-verifier then checks: (a) used_source_ids is non-empty and all IDs correspond to actual retrieved passages; (b) for each cited source, n-gram overlap (starting at 8-word, minimum 4-word, requiring ≥ 20 characters for any matched n-gram) between the answer text and the source content. If the LLM fabricates a source ID or the answer text has insufficient lexical overlap with the cited sources, the verifier returns verified = False and the system returns an abstention response. This two-layer safeguard (prompt constraint + post-verification) ensures the system produces

zero hallucination in practice — every presented answer is traceable to specific legal passages.

7.6 End-to-End Flow

When a user submits a question via the web UI, the server executes six stages before returning the response.

Stage 1 — Topic Classification. The query string is encoded using the TextCNN tokeniser and passed through the TextCNN model to obtain softmax probabilities over the five legal-domain classes. The classification function returns the top three predicted labels with their confidence scores.

Stage 2 — Topic Filter Decision. If the top prediction's confidence meets or exceeds 0.85, the system maps the predicted label through the label merge map and returns a list of corpus-level labels to restrict the search space. If confidence falls below this threshold, all documents are searched without topic-based filtering.

Stage 3 — Hybrid Search. The query is tokenised, transformed into a TF-IDF vector, and compared against the corpus matrix to obtain similarity scores. The top 200 candidates are selected through an efficient partial sort, optionally filtered by topic label indices. The query is then encoded by the Siamese BiLSTM encoder into a fixed-dimensional vector, and cosine similarity is computed against the precomputed document embeddings to produce the final ranked list.

Stage 4 — Relevance Rating. The top ten results are sent to the Groq LLM for relevance scoring on a scale of 0 to 10. The LLM's ratings are attached to each result for later use in answer synthesis.

Stage 5 — Answer Synthesis. The system first runs pre-checks on query length (minimum 4 words), topic confidence (minimum 0.85), and retrieval score (minimum 0.45). It then selects the top five retrieved contexts and sends them to the Groq LLM with a strict source-only prompt that constrains the model to use only verbatim text from the provided passages. The LLM's response is parsed and verified through n-gram overlap checking against the source passages. If verification fails, the system returns an abstention response indicating it cannot answer the query from the available corpus.

Stage 6 — Response Rendering. The server returns a structured JSON response containing the topic prediction, the verified answer (with confidence score and source identifiers), and

the ranked retrieval results (with document names, relevance scores, and domain labels). The browser renders this data into three visual cards: topic prediction, grounded answer, and ranked document results.

Total end-to-end latency is approximately 5 to 15 seconds on CPU, dominated by the two Groq API calls (relevance rating and answer synthesis) which each take 2 to 6 seconds. The Siamese encoding and TF-IDF search together complete in under 1 second.

7.7 Current Limitations

Retrieval limitations. The Siamese BiLSTM achieves MRR = 0.377, meaning the correct passage is the top-1 result only 24.5% of the time. This limits the practical utility of the system: users frequently need to scan multiple results to find the relevant passage. The absence of a cross-encoder re-ranker means the system cannot perform fine-grained pairwise relevance scoring between the query and each candidate. The frozen-word2vec design (CNN-static, not fine-tuned) caps embedding quality at the pretrained language model's capability. For maximum sequence length = 256, passages longer than 256 tokens are truncated, losing tail information. Corpus coverage gaps. The VNLegal dataset does not include foundational codes such as Bộ luật Hình sự (Criminal Code) or Bộ luật Lao động 2019 (Labour Code). Criminal-law and labour-law queries cannot be answered regardless of retrieval quality — the necessary statutes are simply not in the corpus. Only 7 documents contain the word *trộm* (theft), and none are about criminal penalties. The dataset also lacks laws enacted in 2025 or later, limiting coverage of recent legislation. Topic classification limitations. Justice & Dispute Resolution recall is stuck at 31% across all experiments — the class covers 5+ distinct legal sub-domains, and TextCNN cannot learn a consistent feature set for such a diverse label. Without finer-grained labels (e.g., splitting Justice into Criminal Procedure, Civil Procedure, Administrative, Notary), further improvement is unlikely. The frozen Word2vec embeddings are not fine-tuned for legal domain language, leaving a potential gain from domain-specific adaptation (e.g., PhoBERT) on the table. System limitations. The pure-Python HTTP server is single-threaded for model loading (though ThreadingHTTPServer handles concurrent requests). Inference runs on CPU, adding ~5–15 seconds of per-query latency versus <1 second on GPU. The Groq API dependency requires an internet connection and a valid API key; free-tier Groq accounts have rate limits that may throttle heavy usage. There is no session persistence — each query is independent and the system does not maintain conversation history. Evaluation gaps. No BM25 baseline has been

implemented, making it impossible to quantify whether TF‑IDF or BM25 is the better sparse retriever for this domain. No quantitative hybrid comparison (MRR for TF‑IDF‑only vs. Siamese‑only vs. cascaded hybrid) has been run. The answer‑quality evaluation is qualitative (no human‑annotation study or automated answer‑quality metric).

Chapter 8: Conclusion

8.1 Summary

This project built a complete legal retrieval-augmented generation pipeline for Vietnamese statutory law, comprising three integrated stages. Data processing transformed the raw VNLegal dataset (9,582 passages, 56K+ QA pairs, 8 legal domains) into a structured corpus with document-aware stratified splits to prevent data leakage, four class-merging variants (k1–k4) for studying label-granularity effects, and a Vietnamese syllable-level word2vec (979K vocabulary, 300-dim, 3.4 GB) integrated as frozen embeddings with Kim (2014)-conformant OOV initialisation via Uniform[−a, a] where $a = \sqrt{(3 \cdot \sigma^2\_w2v)}$. Model training and evaluation produced eight TextCNN experiments and three Siamese BiLSTM experiments. The TextCNN best configuration (k4, 5-class, dropout probability = 0.5) achieved macro F1 = 0.7602 — a +15.2-point gain over the 8-class baseline — with a confirmed U-shaped dropout curve and evidence that embedding dropout is detrimental for frozen embeddings (−4.4 F1 points). The Siamese BiLSTM best configuration (maximum sequence length = 512, 1 layer) achieved MRR = 0.377, with an inverted-U relationship across sequence lengths (256 < 512 > 1024) explained by the BiLSTM memory-window trade-off. Three architectural adaptations from Neculoiu et al. (2016) were experimentally validated: 1-layer BiLSTM (4-layer collapses for word-level input), masked mean pooling (captures whole-passage context), and a modified contrastive loss with $L^+ = 3.0 \cdot (1-\cos)^2$, $L^- = \text{relu}(\cos-0.5)^2$ (prevents gradient dead zone from margin annealing and balances the 4:1 neg-pos ratio). System deployment packages the pipeline as a single-file web application (480 lines, no external web framework) with TF-IDF first-pass retrieval, Siamese re-ranking, conservative topic filtering (only when confidence ≥ 0.85), and Groq-LLM answer synthesis with automatic abstention via n-gram overlap verification — producing zero hallucination in the deployed system. The repository contains 2,065 lines of source code, 1,674 lines of experiments, 18 Colab notebooks, and production model artifacts.

8.2 Limitations

Technical limitations. (1) Retrieval quality. The Siamese BiLSTM achieves MRR = 0.377 and Recall@1 = 0.245 — the correct passage is the top result less than a quarter of the time. A cross-encoder re-ranker (e.g., MonoBERT, multilingual XLM-R fine-tuned on Vietnamese legal QA) would substantially improve top-1 accuracy by performing pairwise query-passage

scoring rather than relying on fixed embeddings. (2) No BM25 baseline. The sparse retrieval component uses TF-IDF, but BM25's term-frequency saturation and document-length normalisation typically outperform vanilla TF-IDF on retrieval benchmarks. Without a BM25 baseline, the relative contribution of the Siamese embeddings versus the sparse retriever is not fully quantified. (3) Frozen embeddings only. The word2vec embeddings are not fine-tuned during training (CNN-static variant). Kim (2014) showed that fine-tuning (CNN-non-static) yields additional gains, and domain-specific fine-tuning on legal text would likely improve both classification and retrieval. (4) CPU-only inference. The demo runs on CPU, adding 5–15 seconds per query. GPU inference would reduce latency to under 1 second. Data limitations. (5) Missing Criminal Code. Bộ luật Hình sự (Criminal Code) and Bộ luật Lao động (Labour Code) are absent from the VNLegal dataset, making criminal-law and labour-law queries unanswerable regardless of retrieval quality. (6) Justice class diversity. The Justice & Dispute Resolution label encompasses 5+ distinct sub-domains (criminal procedure, civil procedure, notary, enforcement, juvenile justice), capping classifier recall at 31%. (7) Dataset snapshot. The dataset includes laws up to approximately 2024; newer legislation is not covered. Evaluation limitations. (8) No formal hybrid comparison. TF-IDF-only vs. Siamese-only vs. cascaded hybrid have not been quantitatively compared on a held-out test set with MRR/Recall@k. (9) No cost-benefit analysis. The trade-off between Groq API latency/cost and answer-quality improvement over the extractive baseline has not been measured. (10) No human evaluation of answer quality. The grounded-answer verification ensures factual accuracy, but the fluency, coherence, and practical usefulness of the LLM-generated answers have not been evaluated by human annotators.

8.3 Future Work

Immediate priorities (< 1 month). Implement a BM25 baseline using the rank_bm25 package and run a formal comparison of TF-IDF, BM25, Siamese-only, cascaded hybrid, and weighted-fusion hybrid (α sweep from 0.0 to 1.0) with MRR, Recall@k, and MAP as metrics. Fine-tune a cross-encoder re-ranker (multilingual XLM-R or PhoBERT) on the VNLegal QA pairs to re-rank the top-50 Siamese candidates, and measure the MRR improvement. Conduct a controlled experiment comparing question-only vs. question-plus-answer TextCNN training to quantify the distribution-matching benefit identified in Section 6.5. Medium-term improvements (1–3 months). Apply focal loss ($\alpha = 0.75$, $\gamma = 2.0$) to the TextCNN training to address the Justice-class recall bottleneck by down-weighting the well-classified majority

classes. Add Bộ luật Hình sự 2015 (amended) and Bộ luật Lao động 2019 to the corpus to enable criminal-law and labour-law QA. Increase TextCNN maximum sequence length to 384 (current 256 truncates ~17% of answer text) with sequence-length analysis to confirm the trade-off. Implement Mixup data augmentation (Zhang et al., 2018) to reduce the val-test F1 gap caused by overfitting to the validation set. Longer-term directions (3+ months). Replace the TextCNN with a fine-tuned PhoBERT (VinAI, 2021) for topic classification, potentially improving F1 beyond the Kim-style CNN. Extend the single-label TextCNN to multi-label classification — a legal query may span multiple domains (e.g., a question about banking licensing in special economic zones involves both Finance and State organisation). Fine-tune legal-domain word2vec on the VNLegal corpus (or a larger Vietnamese legal corpus) to replace the news-trained vectors with domain-specific embeddings. Replace the Groq API dependency with a locally deployed LLM (e.g., Llama 3.1 8B quantised via llama.cpp or MLC-LLM) for lower latency, offline capability, and zero API costs. Split the Justice label into sub-classes (Criminal Procedure, Civil Procedure, Administrative, Notary, Enforcement) if finer-grained annotations become available.

References

[1] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751. Doha, Qatar. DOI: 10.3115/v1/D14-1181. (Paper summary:.local/papers/textcnn.md)

[2] Neculoiu, P., Versteegh, M., & Rotaru, M. (2016). Learning Text Similarity with Siamese Recurrent Networks. In *Proceedings of the 1st Workshop on Representation Learning for NLP (RepL4NLP) at ACL 2016*, pp. 148–157. Berlin, Germany. (Paper summary:.local/papers/siamese-bilstm.md)

[3] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. In *Advances in Neural Information Processing Systems (NIPS)*, 26, pp. 3111–3119.

[4] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.

[5] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992.

[6] Xiong, L., Xiong, C., Li, Y., Tang, K., Liu, J., Bennett, P., Ahmed, J., & Overwijk, A. (2020). Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *Proceedings of the 2021 International Conference on Learning Representations (ICLR)*.

[7] Robertson, S. & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), pp. 333–389.

[8] Salton, G. & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), pp. 513–523.

[9] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). LEGAL-BERT: The Muppets straight out of Law School. In *Findings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP Findings)*, pp. 2898–2904.

[10] Zheng, L., Guha, N., Anderson, B., Henderson, P., & Ho, D. E. (2021). When Does Pretraining Help? Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset. In *Proceedings of the 18th International Conference on Artificial Intelligence and Law (ICAIL 2021)*, pp. 150–160.

[11] VinAI Research (2021). PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the 2021 Conference of the Association for Computational Linguistics (ACL Findings)*. Diacritic normalisation dictionary released under Apache 2.0 license.

[12] Slobodkin, A., Atsmony, O., & Goldberg, Y. (2023). You Only Prompt Once: On the Capabilities of LLMs for Multi-Label Text Classification. *arXiv preprint arXiv:2310.03132*.

[13] Nogueira, R. & Cho, K. (2019). Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

[14] Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pp. 758–759.

[15] Zhang, H., Cissé, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond Empirical Risk Minimization. In *Proceedings of the 2018 International Conference on Learning Representations (ICLR)*.

[16] Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T., Rosenberg, M., Song, X., Stoica, A., Tiwary, S., & Wang, T. (2016). MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. *arXiv preprint arXiv:1611.09268*.